Computers & Operations Research 176 (2025) 106952

Contents lists available at ScienceDirect

Computers and Operations Research

journal homepage: www.elsevier.com/locate/cor

Mathematical modeling and hybrid evolutionary algorithm to schedule flexible job shop with discrete operation sequence flexibility

Shuai Yuan [a], Xiaomin Zhu [a], Wei Cai [a], Jinsheng Gao [b], Runtong Zhang [c,*]

[a] School of Mechanical, Electronic and Control Engineering, Beijing Jiaotong University, Beijing, China
[b] School of Economics and Management, North China Electric Power University, Beijing, China
[c] School of Economics and Management, Beijing Jiaotong University, Beijing, China

ARTICLE INFO

Keywords:
Flexible job shop scheduling
Mixed-integer linear programming model
Discrete operation sequence flexibility
Hybrid evolutionary algorithm
Knowledge-driven reinitialization
Iterative local search

ABSTRACT

In actual industrial production, several operations of a job may not have precedence relationships and can be placed at any point in the process route. However, traditional flexible job shop scheduling problems (FJSP) often assume that all operations of each job must be processed in strict linear order. Therefore, this research addresses the FJSP with discrete operation sequence flexibility (FJSPDS) with the objective of minimizing the makespan. Based on existing models, two novel mixed-integer linear programming (MILP) models are formulated by improving the description methods of variables and constraints, significantly enhancing the models' performance. Additionally, a hybrid evolutionary algorithm (HEA) is proposed to solve large-scale instances through the following three aspects. An improved encoding method is proposed, which makes the search space of the HEA and solution space of the problem more compatible and reduces the possibility of optimal solutions being missed. A special neighborhood structure is designed according to the characters of sequence-free operations, and an iterative local search method is introduced to improve the quality of the solution. A knowledge-driven reinitialization operator is developed, which generates new individuals based on the features of the historical elite population, guiding the evolution of populations, avoiding premature convergence while also avoiding falling into local optima. Finally, a total of 110 benchmark problem instances are utilized to verify the superior effectiveness of the MILP models and the HEA in solving FJSPDS.

1. Introduction

As an important component of modern manufacturing management, effective scheduling is essential for organizations to improve productivity, reduce costs and ensure their competitiveness in the market (Chen et al., 2023; Xiao et al., 2024). Since the emergence of the Flexible Job Shop Scheduling Problem (FJSP), a great deal of attention has been paid to it by researchers and practitioners due to its computational complexity and relevance in real-life situations (Ghaedy-Heidary et al., 2024; Ji et al., 2023; Li et al., 2022b). The FJSP assumes that the operations of each job are linear-sequence. However, the operations of a job are not always in a linear order, and there may be multiple dependencies among them in some production environments, such as printing(Birgin et al., 2014, Birgin et al., 2015), boarding (Vilcot and Billaut, 2008), glass (Alvarez-Valdes et al., 2005), and the cutting tool industries (Vital-Soto et al., 2020). With this topic in mind, Alvarez-Valdes et al. (2005) study FJSP with some special characteristics, which allow for overlap between operations. Similarly, Vilcot and Billaut (2008) deal with general FJSP that assumes that there may be more than one preceding operation for an operation, but at most one direct successor operation in job routing. Birgin et al. (2014) first consider this problem as an extension of FJSP and propose a mixed-integer linear programming (MILP) model and a set of benchmark problem instances. Subsequently, they propose list scheduling and beam search methods to solve the problem and name it FJSP with sequencing flexibility (FJSPSF) (Birgin et al., 2015). Recently, Gong et al. (2023) research the FJSP considering discrete operation sequence flexibility (FJSPDS), which is a special case of FJSPSF where some operations of a job may not have any sequence constraints.

Inspired by the practical importance and sparse literature, this work tries to study FJSPDS in depth. The traditional FJSP is NP-hard that requires solving two subproblems, i.e., assigning each operation to a machine and sequencing the operations processed by the same machine. In addition to the above two sub problems, the FJSPDS also needs to

* Corresponding author.
E-mail address: rtzhang@bjtu.edu.cn (R. Zhang).

https://doi.org/10.1016/j.cor.2024.106952
Received 5 May 2024; Received in revised form 9 December 2024; Accepted 9 December 2024
Available online 17 December 2024
0305-0548/© 2024 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

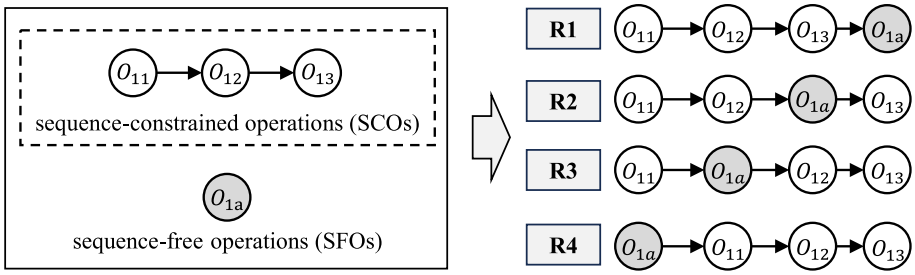


Fig. 1. Optional scheduling schemes to process the job.

Table 1
Summary of related literature.

Problem	Author	Method	Features
JSPSF	Huang et al. (2014)	Improved genetic algorithm	
	Xu et al. (2017)	Bat algorithm	
FJSPSF	Birgin et al. (2014, 2015)	MILP, List scheduling, Beam search	Non sequential constrained operations of a job can be processed simultaneously
	Lin et al. (2023)	MILP, Learning-based cuckoo search algorithm	Non sequential constrained operations of a job can be processed simultaneously
	Cao et al. (2021)	MILP, Knowledge-based cuckoo search algorithm	Mating operations which refer to that two or more operations must be processed together
	Yu et al. (2017)	MILP, Improved differential evolution algorithm	Different priority of jobs
	Liu et al. (2019)	MILP, Genetic algorithm	
	Vital-Soto et al. (2020)	MILP, Hybrid bacterial foraging optimization algorithm	
	Kasapidis et al. (2021)	MILP, Evolutionary algorithm	
	Vital-Soto et al. (2023)	MILP, Genetic algorithm	Dual-resource constraint
FJSPDS	Gong et al. (2023)	MILP, Improved memetic algorithm	
DFJSPSF	Tang et al. (2024)	MILP, Effective memetic algorithm	

JSPSF: the job shop scheduling problem with sequencing flexibility.
DFJSPSF: the distributed flexible job shop scheduling problem with sequencing flexibility.

solve the problem of ordering the sequence-free operations (SFOs) of each job. Fig. 1 displays an example of a job to be processed, in which some operations have strict linear sequence constraints, while others do not have any sequence constraints. For this job, operations O_{11}, O_{12}, and O_{13} must be processed in a linear order, which are called sequence-constrained operations (SCOs), while operation O_{1a} has no sequence constraints with any other operation, meaning it can be arranged at any position along the routing. For the above example, since O_{1a} is an SFO, there are four optional scheduling schemes (R1, R2, R3 and R4), as shown in Fig. 1. Of course, as the number of SFOs increases, there are more optional scheduling schemes for jobs. The feature of discrete operation sequence flexibility enables managers to adjust the operation sequence of jobs based on resource utilization, which not only improves resource utilization but also enhances scheduling flexibility, making the FJSPDS more complex than the classical FJSP. In short, obtaining high robustness and high-quality solutions to the FJSPDS within a reasonable time is extremely challenging and of great significance to manufacturing enterprises.

In order to solve the FJSPDS, an extended version of FJSP, this work makes methodological contributions and proposes new methods. The main contributions of this paper are listed below:

(1) For the FJSPDS, two novel effective MILP models are constructed by improving the description methods of variables and constraints, and its correctness is verified using the Gurobi solver. Compared with the model of Gong et al. (2023), they, especially the second one, can obtain the optimal solutions of the small-scale instances in a shorter time, and for the large-scale instances, they are able to obtain better solutions and improve the lower bound.
(2) For solving large-scale instances, a hybrid evolutionary algorithm (HEA) is presented. The HEA has three main features. First, based on the characters of SFOs, an improved encoding scheme is proposed, which can fully represent all feasible solutions in the solution space and avoid optimal solutions from being excluded from the search space. Second, the iterative local search operator is designed to explore different neighbor search spaces to improve the quality of the solution. Third, a knowledge-driven reinitialization method is proposed to acquire knowledge from elite individuals, which can be used to balance exploration and exploitation and guide population evolution.
(3) Through computational experiments on 110 instances, the superiority of the proposed HEA is demonstrated compared with the exist algorithm in handling FJSPDS, and new best solutions are reported.

The remaining is organized as follows. Section 2 introduces a literature review of related problems. In Section 3, the FJSPDS and the two MILP models are described. In Section 4, HEA incorporating an iterative local search algorithm and a knowledge-driven reinitialization method is designed for the problem. In Section 5, the performance of the MILP models and HEA algorithm are verified experimentally. Finally, the conclusion is given in Section 6.

2. Literature review

To our knowledge, there is limited literature available specifically addressing FJSPDS. Therefore, this section reviews the research on related issues, including FJSP with sequencing flexibility and the application of the evolutionary algorithm to solve FJSP.

2.1. FJSP with sequencing flexibility

Since the introduction of FJSP, it has received a lot of interest from researchers and scholars due to its strong theoretical and application background (Fan et al., 2021; Tamssaouet et al., 2021; Schworm et al., 2024). In recent years, various variants have been extensively studied (Jiang et al., 2021; Destouet et al., 2023; Tutumlu and Saraç, 2023), such as random machine breakdown (Duan and Wang, 2022), multiple resource cooperative constraints (Usman et al., 2024), and lot streaming (Liu et al., 2024b). With the transformation of production modes, the flexibility of production systems has become an increasingly important

focus, and operation sequence flexibility is considered the most important type of flexibility (Beach et al., 2000; Dauzère-Pérès et al., 2024; Weckenborg et al., 2024). Therefore, a few researchers have conducted research on it. Sequencing flexibility refers to the fact that all operations of a job should be processed, but the sequence of some operations may not be fixed (Huang et al., 2014). Sequencing flexibility is a type of soft flexibility that originates from product structure itself rather than hardware such as machines. Birgin et al. (2014, 2015) study the extension of FJSP, in which the precedence between operations is described by arbitrary directed acyclic graph (DAG) instead of a simple linear order, and explicitly named it FJSP with sequencing flexibility. Liu et al. (2019) referrer to the shop floor considering sequencing flexibility as an ultra-flexible job shop (u-FJS).

Table 1 summarizes the literature related to FJSPSF, including problem types, solving methods, and features. Huang et al. (2014) address the job shop scheduling problem with sequencing flexibility (JSPSF), and propose an improved genetic algorithm with the redesigned coding scheme. Xu et al. (2017) devise a bat algorithm hybrid with genetic algorithm and propose a linearly decreasing inertia weighting strategy to overcome the disadvantage of fixed parameters. Birgin et al. formulate a MILP model with makespan criteria, introduce a list scheduling algorithm, and develop a beam search method to address the FJSPSF (Birgin et al., 2014, Birgin et al., 2015). Lin et al. (2023) develop a learning-based cuckoo search algorithm to minimize the makespan of FJSPSF, and the experiment results show that it outperforms other algorithms. However, their models do not take into account the constraint that at most one operation of a job can only be processed by one machine at the same time. Cao et al. (2021) concentrate on the FJSPSF with mating operations, which refers to that two or more operations must be processed together. They propose a knowledge-based cuckoo search algorithm with reinforcement learning and hybrid heuristics to this scheduling field. Yu et al. (2017) develop an improved differential evolution algorithm with local search to solve the FJSPSF with priority. Other methods, such as genetic algorithm (Liu et al., 2019; Vital-Soto et al., 2023), hybrid bacterial foraging optimization algorithm (Vital-Soto et al., 2020) and evolutionary algorithm (Kasapidis et al., 2021), are introduced to address the FJSPSF. Tang et al. (2024) formulate a MILP model for the distributed flexible job shop scheduling problem with sequence flexibility (DFJSPSF) and propose an effective memetic algorithm. Gong et al. first consider the FJSPDS, propose a MILP model and design an improved memetic algorithm.

2.2. Evolutionary algorithm for FJSP

Inspired by the idea of biological evolution, evolutionary algorithm (EA), an important type of *meta*-heuristic algorithms, has been proposed and widely applied to combinatorial optimization problems (Liang et al., 2023; Zhang et al., 2023; Lu and Lu, 2024), because it can adjust the encoding method according to the characteristics of the problem and obtain better solutions in a shorter time. The four commonly used encoding strategies are as follows: binary-alphabet-based strategy (Chen et al., 1999), machine-based assignment and operation sequences (Jia and Hu, 2014), unified encoding strategy (Sun et al., 2021; Xu et al., 2024), and machine assignment and operation sequence (MAOS) (Burmeister et al., 2023; Chen et al., 2023b; Sun et al., 2023; Zhou et al., 2024). Among these four strategies, the fourth one is the most widely used because it is easier to design operators to generate new solutions and integrate local search operators to obtain better operation sequence, and its discrete characteristics are naturally consistent with the evolutionary algorithm (Gao et al., 2019).

In the general framework of EA, crossover and mutation operators are the main ways to generate new individuals. For the crossover operations, single point crossover (1PX), two-point crossover (2PX), uniform crossover (UX), and precedence operation crossover (POX) are commonly employed, and for the mutation operations, single point mutation (1PM), two-point mutation, swap mutation, and local search are usually used (Neumann et al., 2023). The algorithm utilizes MAOS strategy, UX, POX, and 2PX as crossover operators, as well as local search as mutation operators. Duan and Wang (2021) use non-dominated sorting genetic algorithm (NSGA-II) to solve the FJSP with variable machine speed. On the basis of the MAOS encoding strategy, population updates are carried out using IPX, multi-point crossover, 1PM, and Gaussian mutation. Gong et al. (2021) propose a non-dominated ensemble fitness ranking method to solve FJSP with worker flexibility and green factors. Due to the consideration of workers, the vector of worker assignment is added to MAOS encoding, and POX, random probability crossover, and 1PM are used as genetic operators. Liu et al. (2021) develop a hybrid genetic algorithm based on the unified encoding strategy to solve FJSP, use UX as the crossover operator, and introduce particle swarm optimization algorithm as the mutation operator. Pan et al. (2022) propose a bi-population evolutionary algorithm for FJSP, MAOS, UX, POX, generalized position crossover, insert mutation, swap mutation, and inverse mutation are adopted. An et al. (2023) deal with the multi-objective FJSP by using the evolutionary algorithms, in which MAOS, multi-point crossover, and POX are employed. Saouabi et al. (2024) propose a two-level evolutionary algorithm for dynamic FJSP, where job based crossover and swap mutation based on MAOS encoding are applied.

At present, to solve FJSP effectively and efficiently, local search strategies are proposed to improve the convergence speed and local optimal searching performance of EA (An et al., 2020; Guo et al., 2024; Liu et al., 2024a; Shao et al., 2025). An et al. (2020) employe a multi-objective evolutionary algorithm that integrates variable neighborhood search and tabu search to solve the FJSP scheduling problem considering forklift transportation and imperfect tool maintenance. In order to jump out of local optima of EA, Li et al. (2022a) and Liu et al. (2024a) propose a variable neighborhood search fixed with multi local search methods for FJSP. Guo et al. (2024) introduce a multiple neighborhood search to perform a local search around the current individual generated by EA. Shao et al. (2025) propose a special local search strategy based on the principle of job balancing the rational selection of the number of machines and jobs allocation for FJSP with lot streaming.

In summary, to address the current shortcomings of the existing work about FJSPDS, this paper constructs more efficient MILP models for the problem and proposes a HEA algorithm with better performance to realize the efficient utilization of production resources in the shop floor. Finally, extensive experiments are conducted on the benchmark instances, and the results proved the superiority of the proposed algorithm compared to existing algorithms.

3. Problem formulation

3.1. Problem description

The FJSPDS is described as follows:

There are n jobs $J = \{J_1, J_2, \ldots, J_n\}$ need to be processed by m machines $M = \{M_1, M_2, \ldots, M_m\}$ in the shop. The job J_i contains some SCOs $O_i^c = \{O_{i1}, O_{i2}, \ldots, O_{i,ci}\}$ and SFOs $O_j^f = \{O_{ia}, O_{ib}, \ldots, O_{i,fi}\}$. Each operation could be processed by several machines whose processing time depends on the machine. FJSPDS consists of two subtasks: (1) Machine selection. Determine the machine selection of each operation according to their available machines. (2) Operation scheduling. Determine the operation sequence on each machine with consideration of the discreate operation sequence flexibility.

In addition to the above, some assumptions are made as follows:

(1) All machines and jobs are available at time 0.
(2) Each machine can process only one job at a time, and each job can only be processed by one machine at a time.

Table 2
An example of FJSPDS with 3 machines and 3 jobs.

Job	Operation	Processing time		
		M_1	M_2	M_3
J_1	O_{11}	36	–	40
	O_{12}	56	45	–
	O_{1a}	–	–	23
	O_{1b}	45	56	–
J_2	O_{21}	–	34	28
	O_{22}	68	56	53
	O_{23}	–	48	–
	O_{2a}	54	–	–
J_3	O_{31}	34	–	39
	O_{32}	56	50	–
	O_{33}	78	57	–
	O_{34}	–	34	39

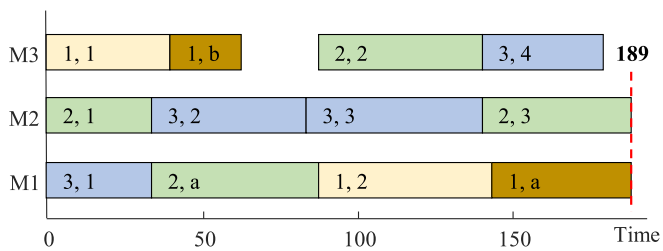


Fig. 2. An optimal schedule for the example.

Table 3
. Notation used for MILP-1.

m	Parameters
	The number of jobs
	The number of machines
i, h	The index for jobs, i, $h = 1, 2, \ldots, n$
n_j	The number of operations of job j
j, e	The index for operations, j, $e = 1, 2, \ldots, n_j$
k, v	The index for machines, k, $v = 1, 2, \ldots, m$
O_{ij}	The j-th operation of job i
O_i	The operation set of job i
M_{ij}	The available machine set for operation O_{ij}
pt_{ijk}	The processing time of operation O_{ij} processed by machine k
a_{ijk}	If operation O_{ij} can be processed by machine k, $a_{ijk} = 1$. Otherwise, 0.
b_{iej}	If operation O_{ij} must be processed before operation O_{ej}, $b_{iej} = 1$. Otherwise, 0.
L	A sufficiently large positive number
Variables	
C_{max}	The maximum completion time (makespan)
st_{ij}	The starting time of operation O_{ij}
ft_{ij}	The finish time of operation O_{ij}
x_{ijk}	Binary variable, if operation O_{ij} is assigned to machine k, $x_{ijk} = 1$. Otherwise, 0.
y_{kivej}	Binary variable, if operation O_{ie} is processed on machine v after operation O_{ij} is processed on machine k, $y_{kivej} = 1$. Otherwise, 0.
z_{ijhek}	Binary variable, if operation O_{he} is processed after operation O_{ij} is processed on machine k, $y_{kivej} = 1$. Otherwise, 0.

(3) For each operation, once it begins to be processed, it cannot be interrupted.
(4) All the processing data such as processing time is deterministic.
(5) SCOs need to be processed according to the strict linear sequence, while SFOs can be placed anywhere in the process route as appropriate.

Table 2 shows an example of a 3 × 3 FJSPDS. There are 3 machines $\{M_1, M_2, M_3\}$, 3 jobs $\{J_1, J_2, J_3\}$, which include 12 operations in total. The jobs J_1 and J_2 involve 2 SFOs $\{O_{1a}, O_{1b}\}$ and 1 SFO $\{O_{2a}\}$, respectively. As shown in Fig. 1, J2 has four feasible operation sequences, while J_1 has twelve operation sequences due to it has more SFOs. As shown in Fig. 2, there is an optimal schedule for the example.

3.2. MILP-1 model for FJSPDS

The mathematical model MILP-1 formulated in this section is an adaptation of the MILP model developed by Gong et al. (2023). The major difference lies in the description method of the precedence relationship of two operations processed on the same machine and the relationship of the start and end times of the same operation. A comparison between the two MILP models for the FJSPDS is provided in Section 5.1. The notation used for MILP-1 is presented in Table 3.

Given above parameters and variables, MILP-1 is formulated.

$$f = minC_{max} \tag{1}$$

Subject to:

$$C_{max} \geq ft_{ij}, \forall i, j \tag{2}$$

$$st_{ej} \geq ft_{ij} - L(1 - b_{iej}), \forall i, j, e : j \neq e \tag{3}$$

$$ft_{ij} \geq st_{ij} + \sum_{k \in M_{ij}} pt_{ijk} x_{ijk}, \forall i, j \tag{4}$$

$$st_{ij} \geq ft_{ej} - L\left(3 - y_{kjvei} - x_{ijk} - x_{iev}\right), \forall i, j, e, k, v : k \in M_{ij}, v \in M_{ie}, j \neq e \tag{5}$$

$$st_{ej} \geq ft_{ij} - L\left(2 + y_{kjvei} - x_{ijk} - x_{iev}\right), \forall i, j, e, k, v : k \in M_{ij}, v \in M_{ie}, j \neq e \tag{6}$$

$$st_{ij} \geq ft_{eh} - L(3 - z_{ijhek} - x_{ijk} - x_{hek}), \forall i, h, j, e, k : k \in M_{ij} \cap M_{he} \tag{7}$$

$$st_{eh} \geq ft_{ij} - L(2 + z_{ijhek} - x_{ijk} - x_{hek}), \forall i, h, j, e, k : k \in M_{ij} \cap M_{he} \tag{8}$$

$$\sum_{k \in M_{ij}} x_{ijk} = 1, \forall i, j \tag{9}$$

$$x_{ijk} \leq a_{ijk}, \forall i, j, k : k \in M_{ij} \tag{10}$$

$$y_{kjvei} \geq b_{iej}, \forall i, j, e, k, v : k \in M_{ij}, v \in M_{ie}, j \neq e \tag{11}$$

$$st_{ij} \geq 0, \forall i, j \tag{12}$$

$$x_{ijk} \in \{0, 1\}, \forall i, j, k : k \in M_{ij} \tag{13}$$

$$y_{kjvei} \in \{0, 1\}, \forall i, j, e, k, v : k \in M_{ij}, v \in M_{ie}, j \neq e \tag{14}$$

$$z_{ijhek} \in \{0, 1\}, \forall i, h, j, e, k : k \in M_{ij} \cap M_{he} \tag{15}$$

Constraint (1) represents the objective function that minimizes the makespan. Constraint (2) ensures that the makespan is no less than the completion time of all operations instead of the last operation of all the jobs, because the last operation for each job cannot be predetermined due to the presence of SFOs. Constraint (3) represents the precedence relationship of all operations of the job. Constraint (4) restricts that the operation cannot be interrupted when processing. Constraints (5) and (6) show that only one operation of each job can be processed at a time. Constraints (7) and (8) define that the process times on the same machine do not overlap, ensuring that a machine can only process one job at a time. Constraints (9) and (10) guarantee that each operation can only be processed on a machine. Constraint (11) enforces that all SFOs must follow the process route being processed. Constraints (12), (13), (14) and (15) indicate the range of values of the decision variables.

3.3. MILP-2 model for FJSPDS

The mathematical formulation presented in this section is inspired by the earlier works of Yao et al. (2023). However, the main difference between MILP-1 and MILP-2 is that MILP-2 adopts a single index for indexing the operations that leads to a simpler notation. A comparison between the two MILP models for the FJSPDS is provided in Section 5.1.

Table 4
. Notation used for MILP-2.

I	The set of operations
Parameters	
n	The number of jobs
m	The number of machines
h	The index for jobs, $h = 1, 2, \ldots, n$
i, j	The index for operations, $i, j \in I$
k, v	The index for machines, $k, v = 1, 2, \ldots, m$
O_h	The operation set of job h
M_i	The available machine set for operation i
pt_{ik}	The processing time of operation i processed by machine k
a_{ik}	If operation i can be processed by machine k, $a_{ik} = 1$. Otherwise, 0.
b_{ij}	If operation i must be processed before operation j, $b_{ij} = 1$. Otherwise, 0.
L	A sufficiently large positive number
Variables	
C_{max}	The maximum completion time (makespan)
st_i	The starting time of operation i
ft_i	The finish time of operation i
x_{ik}	Binary variable, if operation i is processed on machine k, $x_{ik} = 1$. Otherwise, 0.
y_{ijk}	Binary variable, if operation j is processed after operation i is processed on machine k, $y_{ijk} = 1$. Otherwise, 0.
z_{ij}	Binary variable, if operation j is processed after operation i, $z_{ij} = 1$. Otherwise, 0.

The notation used for MILP-2 is presented in Table 4.

Based on the above parameters and variables, MILP-2 is formulated. Similarly, the objective function of MILP-2 is shown in constraint (1), which also minimizes the makespan. The specific constraints are as follows

$$C_{max} \geq ft_i, \forall i \in I \tag{16}$$

$$st_j \geq ft_i - L\left(3 - y_{ijk} - x_{ik} - x_{jk}\right), \forall i, j \in I, k \in M_i \cap M_j, i \neq j \tag{17}$$

$$st_i \geq ft_j - L\left(2 + y_{ijk} - x_{ik} - x_{jk}\right), \forall i, j \in I, k \in M_i \cap M_j, i \neq j \tag{18}$$

$$st_i \geq ft_j - L\left(1 - z_{ij}\right), \forall i, j \in O_h, i \neq j \tag{19}$$

$$ft_i \geq st_i + \sum_{k \in M_i} pt_{ik} x_{ik}, \forall i \in I \tag{20}$$

$$z_{ij} + z_{ji} = 1, \forall i, j \in O_h, i \neq j \tag{21}$$

$$z_{ij} \geq b_{ij}, \forall i, j \in O_h, i \neq j \tag{22}$$

$$x_{ik} \leq a_{ik}, \forall i \in I, k \in M_i \tag{23}$$

$$\sum_{k \in M_i} x_{ik} = 1, \forall i \in I \tag{24}$$

$$st_i \geq 0, \forall i \in I \tag{25}$$

$$x_{ik} \in \{0, 1\}, \forall i \in I, k \in M_i \tag{26}$$

$$y_{ijk} \in \{0, 1\}, \forall i, j \in I, k \in M_i \cap M_j, i \neq j \tag{27}$$

$$z_{ij} \in \{0, 1\}, \forall i, j \in O_h, i \neq j \tag{28}$$

Constraint (16) ensures that the makespan is no less than the completion time of all operations. Constraints (17) and (18) ensure that the process times on the same machine do not overlap, ensuring that a machine can only process one job at a time. Constraint (19) represents the precedence relationship of all operations of the job. Constraint (20) restricts that the operation cannot be interrupted when processing. Constraints (21) and (22) describe that the operations of the same job must follow the process route and be processed one after another. Constraints (23) and (24) restrict that each operation can only be processed on a machine. Constraints (25), (26), (27) and (28) indicate the range of values of the decision variables.

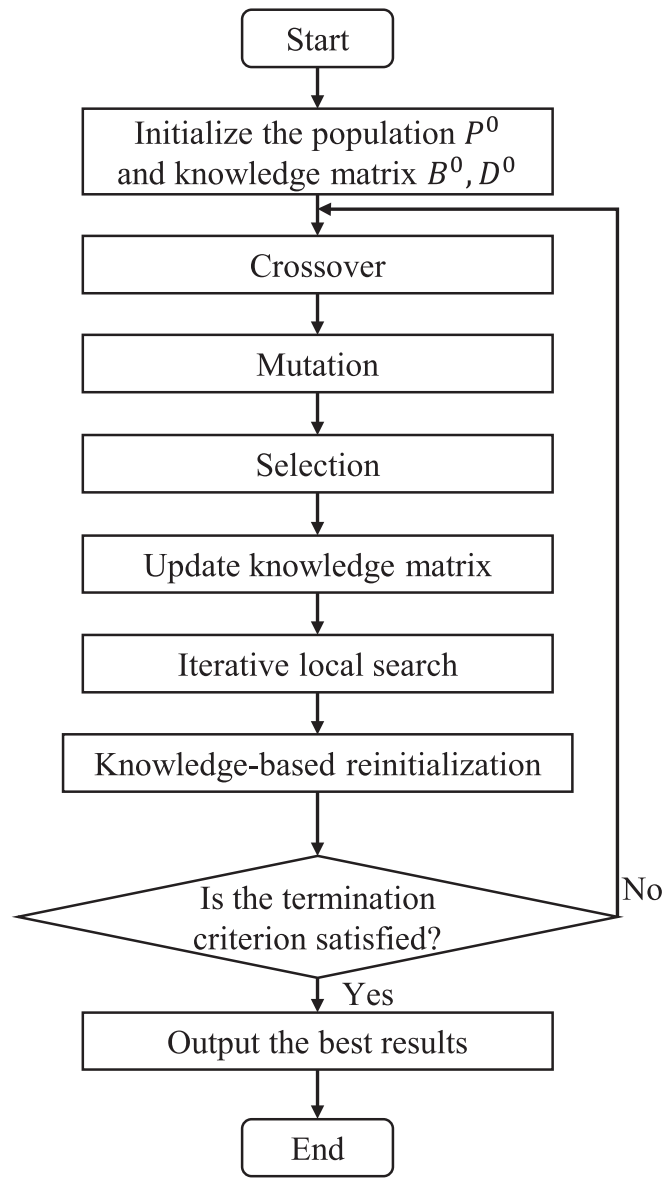


Fig. 3. The workflow of the HEA.

4. The proposed HEA algorithm

4.1. Workflow of the HEA algorithm

The HEA presented in this section is based on the basic framework of evaluation algorithm but incorporates improvements tailored to the characteristics of discrete flexible operations. It mainly includes an improved encoding scheme, an iterative local search method and a knowledge-based reinitialization strategy. The overall workflow of the HEA algorithm is shown in Fig. 3.

Step 1: Initialize the knowledge matrix and the population.

Step 2: Generate the offspring based on the crossover, mutation and selection operators.

Step 3: Update the knowledge matrix based on the features of the elite individuals.

Step 4: Select the elite individuals and perform iterative local search to reinforce them.

Step 5: Generate new individuals based on the knowledge matrix to replace underperforming individuals in the offspring.

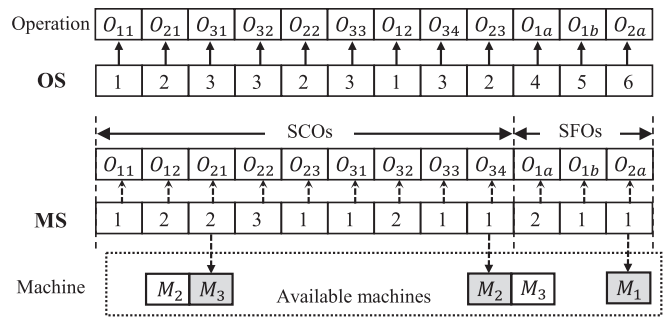


Fig. 4. An example of the solution representation for the instance in Table 2.

Step 6: If the termination condition is satisfied, output the best solution, otherwise, return to Step 2.

4.2. Population initialization

Based on the encoding scheme proposed by Gong et al. (2023) (ES-G), an improved encoding scheme (IES) is proposed in this section. The right-leaning decoding method based insertion decoding method is applied, which can generate feasible and efficient scheduling Gong et al. (2023).

4.2.1. Improved encoding scheme

Each individual consists of two vectors of length N (the total number of operations): the operation sequence (OS) vector and the machine selection (MS) vector. According to the data in Table 2, an individual is shown in Fig. 4. In the OS vector, the first 9 genes indicate the SCOs, each coding number presents the job number, and the number of appearances indicates its operation number; the last 3 genes indicate the SFOs, and each coding number is the total number of jobs plus the sequential number of this sequence flexible operation. Therefore, the operation sequence represented by the OS vector {1, 2, 3, 3, 2, 3, 1, 3, 2, 4, 5, 6} shown in Fig. 4 is $\{O_{11}, O_{21}, O_{31}, O_{32}, O_{22}, O_{33}, O_{12}, O_{34}, O_{23}, O_{1a}, O_{1b}, O_{2a}\}$. The coding numbers 4, 5, and 6 represent O_{1a}, O_{1b}, and O_{2a}, respectively. The MS vector lists the machine numbers in ascending order of jobs for SCOs and SFOs respectively. In the MS vector, the second "2" implies that O_{21} is processed on the second optional machine, which is M_3, the fifth "1" represents O_{34} is processed on the first optional machine, which is M_2, and the last "1" implies that O_{2a} is processed on the first optional machine, which is M_1.

The difference between IES and ES-G lies in whether the SFOs of the same job is encoded with different numbers. Take the instance in Table 2 as an example to illustrate the superiority of the proposed IES. Fig. 5 shows its two individuals and their corresponding Gantt charts based on ES-G and IES encoding methods. As shown in Fig. 5(a), it is a feasible solution with a makespan value of 212. Operation O_{1a} and O_{1b} are encoded with the same number "4", which means that operation O_{1a} must be processed before O_{1b} by default. Therefore, the optimal solution with a makespan value of 189 shown in Fig. 5(b) cannot be obtained. Based on the above example, it can be seen that ES-G prevents the search space of the algorithm from completely covering the solution space of the problem, resulting in the omission of the optimal solution.

4.2.2. Population initialization

The quality of the initial population has a significant impact on the convergence speed and optimization ability of the search. A hybrid initialization method is employed. The OS vector is generated by randomly disordering all operations. The MS vector employs a hybrid method (Shi et al., 2009) combining global selection, local selection and random selection. The probability of applying for the three selection methods is 60 %, 30 % and 10 %, respectively.

4.3. Evolutionary operators

4.3.1. Crossover operator

In this study, an order-based crossover (OBX) operator is adopted for the OS vector and a uniform crossover (UX) operator is employed for the MS vector, and the probability of both crossover operators being selected is 50 %, respectively (Gong et al., 2023).

The procedure of OBX is shown in Fig. 6. The specific steps are as follows:

Step 1: Assuming that l is 1/3 of the length of the vector (if l is not an integer, rounded up to an integer), randomly select l positions on the OS vector as crossover points. In Fig. 5., $l = 4$ and the positions of the crossover points are $Q_{pos1} = [3, 7, 8, 11]$.

Step 2: In individual P1, the gene segments are $S1 = [5, 3, 2, 3]$, and the corresponding operations are $W_{S1} = [O_{1b}, O_{31}, O_{23}, O_{33}]$ according to the coding method described in the Section 4.2.1. In P2, the positions of

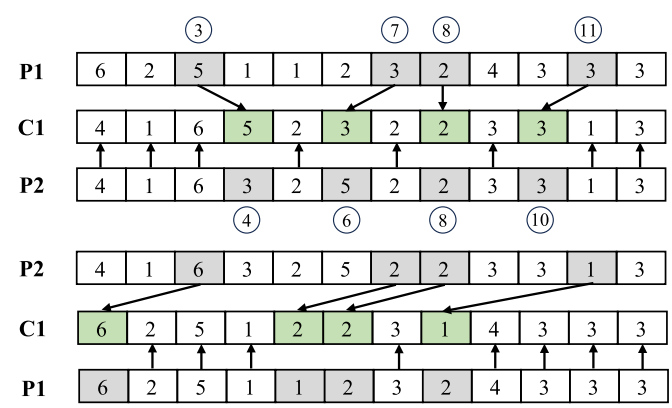
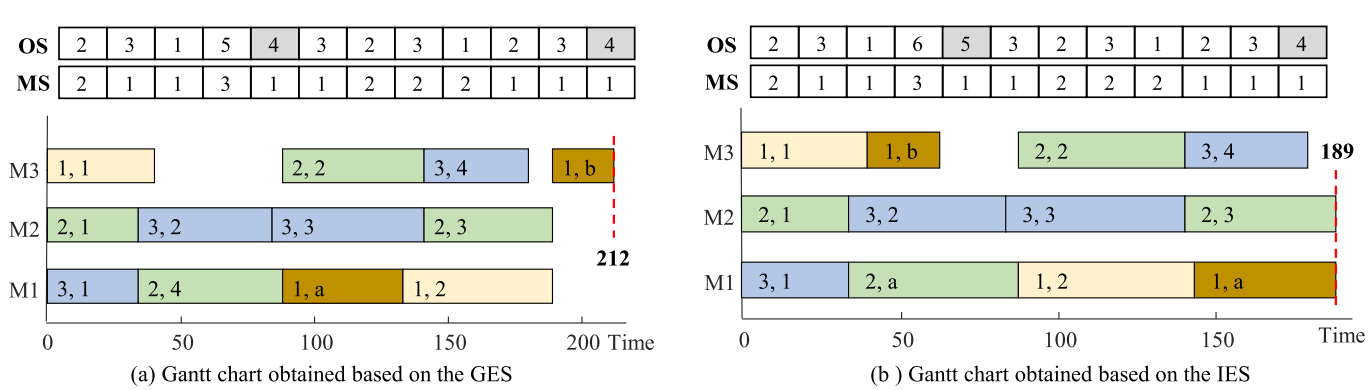


Fig. 6. Schematic diagram of OBX operator for OS vector.



(a) Gantt chart obtained based on the GES (b) Gantt chart obtained based on the IES

Fig. 5. Individuals and their Gantt charts based on different encoding methods.

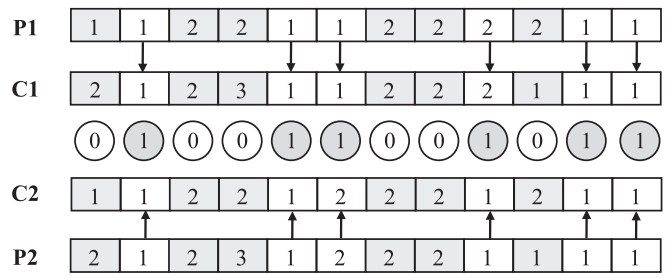


Fig. 7. Schematic diagram of UX operator for MS vector.

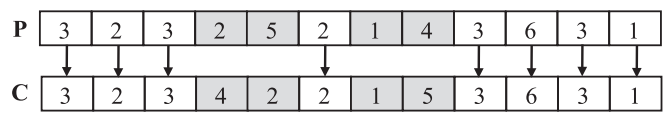


Fig. 8. Schematic diagram of the HMM for OS vector.

operations W_{S1} are $Q_{pos2} = [4, 6, 8, 10]$, the coding numbers except Q_{pos2} in P2 is assigned to C1 at the same position of P2, and then $S1$ is copied to the empty positions of C1 in order.

Step 3: Based on similar operations as in Step 2, C2 can be obtained.

The procedure of UX is presented in Fig. 7. It functions as follows:

Step 1: Randomly generate a 0–1 vector with a length equal to the length of the MS vector.

Step 2: For positions equal to 0 in the 0–1 vector, exchange the value in the same locations in the MS vectors of P1 and P2.

4.3.2. Mutation operator

The mutation operator generates new solutions by introducing random perturbations (randomly modifying the coding vectors), thus allowing the algorithm to jump out of the local optima and avoid premature convergence. The heterotopic mutation method (HMM) is adopted for OS vector, and machine-load balancing mutation method (MBM) is employed for MS vector, and both mutation operators are applied with a probability of 50 % (Gong et al., 2023).

The procedure of HMM is shown in Fig. 8. The new OS vector is obtained by reordering several operations chosen randomly. The specific steps are as follows:

Step 1: Assuming that l is 1/3 of the length of the vector (if l is not an integer, rounded up to an integer), randomly select l positions on the OS vector as the mutation points. In Fig. 8, $l = 4$ and the positions of the mutation points are $Q_{pos} = [4, 5, 7, 8]$.

Step 2: In P, the gene segments are $S1 = [2, 5, 1, 4]$, and the new gene segments $S2 = [4, 2, 1, 5]$ are obtained by randomly sorting $S1$.

Step 3: Copy the genes except Q_{pos} in P to C at the same positions, and $S2 = [4, 2, 1, 5]$ fill into the empty positions of C.

The procedure of MBM is shown in Fig. 9. The machine with maximum completion time is selected, an operation it undertakes is randomly chosen. Choose machines from its optional machine set one by one to replace the current machine, and finally select the machine with

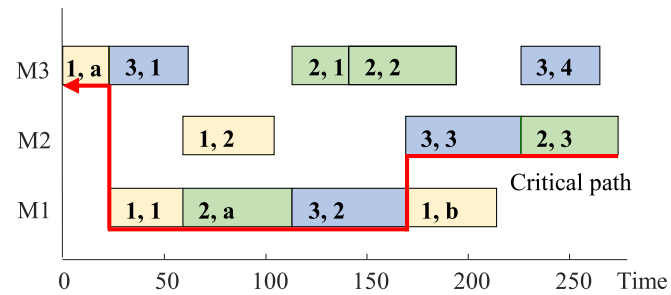


Fig. 10. Schematic of the critical path.

minimum makespan. The specific steps are as follows:

Step 1: Select the machine with the maximum completion time. In Fig. 9, the machine M_2 is selected.

Step 2: Randomly select an operation processed on the machine, which can be processed by more than one machine. The operation selected in the example is O_{22} and its available machines are $\{M_1, M_2, M_3\}$.

Step 3: Choose the machine in the set one by one, calculate the makespan values under different assignment schemes, and select the scheme with the smallest makespan value.

Step 4: Repeat steps 1–3 once.

4.3.3. Selection operator

The strategies of the elite retention and binary tournament selection are employed. Retain the top 10 % of individuals with fitness values from the parent generation directly to the offspring. For the binary tournament selection, the probability of a better or worse individual being selected is 80 % and 20 %, respectively.

4.4. Iterative local search operator

The local search based on the critical path can further improve the search efficiency of the algorithm. A special neighborhood structure is designed based on the characteristics of SFOs, which can be processed at any position in the process route. In addition, an iterative local search operator is proposed by mixing two other classical neighborhood structures in FJSP.

4.4.1. Neighborhood structure

The critical path is the longest process path obtained under the condition of satisfying the process route constraints, and the operations contained in the critical path are labeled as critical operations. Fig. 10 shows the Gantt chart corresponding to one solution of the example shown in Table 2. The operation O_{23} is the last completed operation, and its completion time is makespan. As shown in Fig. 10, the critical path is $\{O_{1a}, O_{11}, O_{2a}, O_{32}, O_{33}, O_{23}\}$. On a machine, a few operations that are processed consecutively and have no idle time are considered as a critical block, such as $\{O_{33}, O_{23}\}$ and $\{O_{11}, O_{2a}, O_{32}\}$. The following are the three neighborhood structures applied in this paper:

(1) NS_1: Randomly select a critical operation and traverse all its

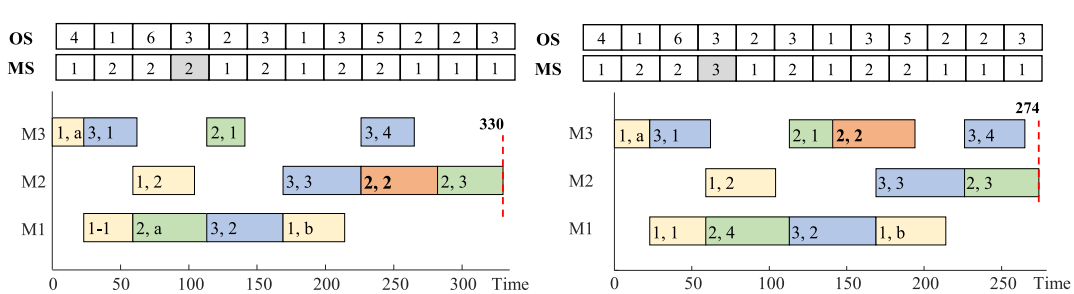


Fig. 9. Schematic diagram of MBM for MS vector.

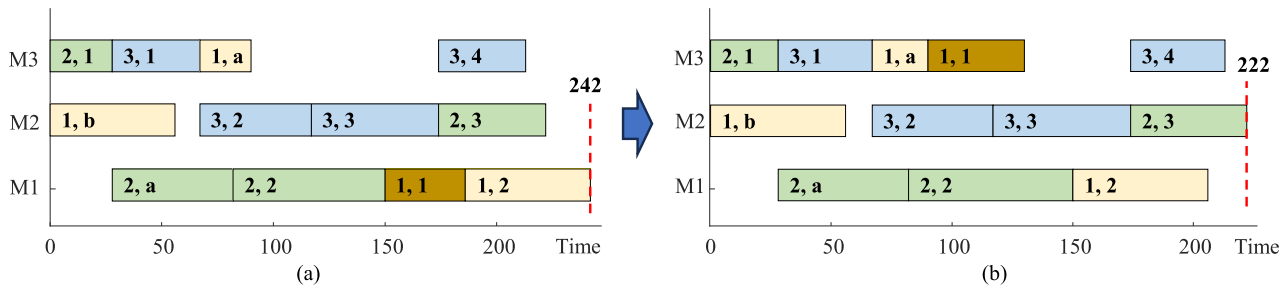


Fig. 11. Example of neighborhood structure NS_1 : move critical operation O_{11} from M_1 to M_3.

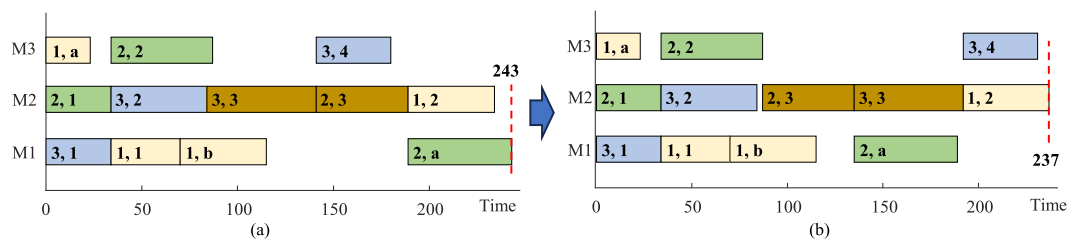


Fig. 12. Example of neighborhood structure NS_2: insert critical operation O_{23} before O_{33}.

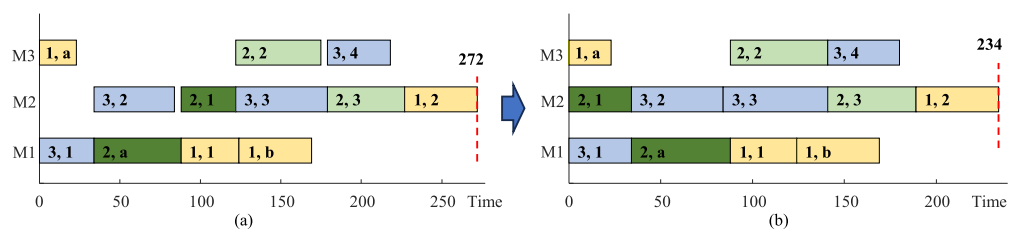


Fig. 13. Example of neighborhood structure NS_2: swap critical operation O_{21} with O_{2a}.

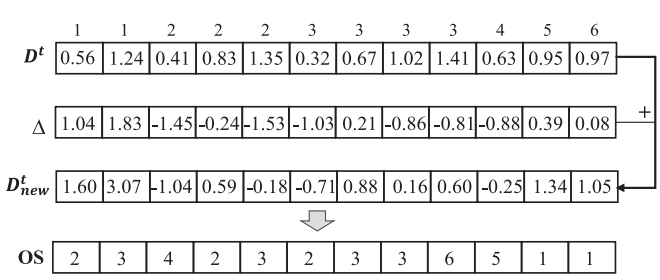


Fig. 14. An example of OS vector generation.

available machines. In Fig. 11, the critical operation O_{11} is assigned to the machine M_3.

(2) NS_2: For a critical block, randomly select a critical operation, and if it is the first operation of the block, then randomly insert it after any operation of the critical block; if it is the last of the block, then randomly insert it before any operation of the critical block; otherwise, then insert it before the first or after the last of the block. In Fig. 12, operations O_{21}, O_{32}, O_{33}, O_{23} form a critical block, and operation O_{33} is inserted after block end O_{23}.

(3) NS_3: In the critical path, check for the presence of jobs that contain SFOs, and if so, then randomly select a sequence-constrained critical operation to be swapped with a sequence-free non-critical operation. In Fig. 13, the critical operation O_{21} is exchanged in position

Table 5
. The size complexity of MILP models.

Instances	MILP-G			MILP-1			MILP-2		
	NCs	NIVs	NBVs	NCs	NIVs	NBVs	NCs	NIVs	NBVs
MPSF01	187	93	80	188	98	85	**138**	**74**	**61**
MPSF02	586	256	233	533	259	237	**294**	**159**	**137**
MPSF03	1070	510	477	980	497	468	**660**	**352**	**323**
MPSF04	3257	1510	1446	3093	1527	1486	**1584**	**851**	**810**
MPSF05	1829	844	796	1628	815	775	**1050**	**562**	**522**
MPSF06	1733	821	771	1540	789	748	**1158**	**622**	**581**
MPSF07	1760	821	763	1637	837	804	**1278**	**699**	**666**
MPSF08	6884	3100	2995	6362	3060	2995	**4413**	**2291**	**2226**
MPSF09	12,211	5861	5717	11,658	5800	5709	**8212**	**4235**	**4144**
MPSF10	11,655	5560	5417	11,157	5494	5443	**8087**	**4148**	**4097**
Mean	4117.2	1937.6	1869.5	3877.6	1917.6	1875	**2687.4**	**1399.3**	**1356.7**

Table 6
. The computational results of MILP models.

Instances	J-M−C−F	MILP-G				MILP-1				MILP-2			
		C_{max}	Time(s)	Gap (%)	LB	C_{max}	Time(s)	Gap (%)	LB	C_{max}	Time(s)	Gap (%)	LB
MPSF01	3–2-7–2	237	0.05			237	0.06			237	**0.04**		
MPSF02	3–3-9–3	189	0.2			189	**0.08**			189	0.09		
MPSF03	4–3-11–5	144	2.12			144	2.02			144	**0.77**		
MPSF04	4–5-12–8	152	159.38			152	10.32			152	**2.11**		
MPSF05	5–4-14–6	144	7.16			144	3.8			144	**2.37**		
MPSF06	7–4-14–7	148	654.07			148	526.61			148	**166.73**		
MPSF07	8–5-10–6	101	19.8			101	48.56			101	**8.11**		
MPSF08	8–5-24–8	**191**	3600	31.41	131	**191**	3600	26.7	140	**191**	3600	**24.61**	**144**
MPSF09	9–5-26–19	281	3600	57.3	120	288	3600	51.74	139	**280**	3600	**48.21**	**145**
MPSF10	10–5-29–21	572	3600	59.09	234	572	3600	51.57	277	**571**	3600	**27.67**	**413**

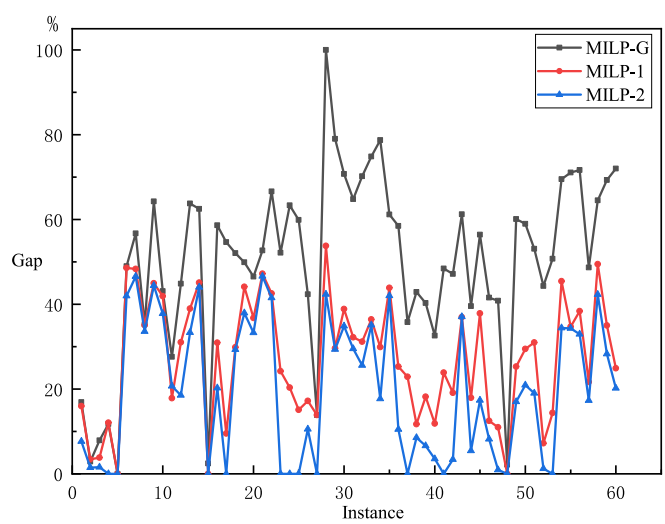


Fig. 15. Gap of instances PSFs 01–60 solved by MILP models.

with an SFO O_{24} belonging to the same job.

4.4.2. Iterative local search

The specific implementation of iterative local search is shown in Algorithm 1. When a new individual S that is better than the current individual P is obtained based on the current neighborhood structure NS_k, go back to the neighborhood structure NS_1 and begins the search anew with S as the starting point. When no better solution is obtained based on the current neighborhood structure, the algorithm jumps to the next neighborhood structure for searching until all neighborhood structures fail to generate a better solution.

Algorithm 1: Iterative local search

Input: Individual P, neighborhood structures NS_1, NS_2 and NS_3
Output: New individual S

```
Initialize, k = 1, k_max = 3
While k <= k_max
  S = NS_k(P)
  If C_m(S) < C_m(P)
    P = S
    k = 1
  Else
    k = k + 1
  End
End
```

4.5. Knowledge-driven reinitialization operator

During the iterative search of evolutionary algorithms, some problems may arise, such as reduction in the diversity of solutions, premature convergence, etc., which may degrade the performance. Although the use of random initialization method can solve these problems to some extent, it still cannot well balance the two aspects of exploration and exploitation. On the other hand, since the evolutionary algorithm generates many individuals in each iteration, the performance of the algorithm can be improved if it can learn the features of the elite individuals, thus guiding the generation of individuals in the re-initialization, and reducing the blindness of random generation.

In this section, a knowledge-driven reinitialization method is proposed. The features of elite individuals about machine selection and operation prioritizing are summarized to guide the generation of new individuals. This section first defines the knowledge, then describes the initialization and updating of the knowledge, and finally gives the method to generate new individuals based on knowledge.

Table 7
. The main parameters of the HEA.

Parameters	Factor level				
	1	2	3	4	5
Cr	0.95	0.9	0.85	0.8	0.75
Mu	0.25	0.2	0.15	0.1	0.05
Es	0.25	0.2	0.15	0.1	0.05
Le	0.25	0.2	0.15	0.1	0.05
Ls	0.5	0.4	0.3	0.2	0.1
Ri	0.25	0.2	0.15	0.1	0.05

4.5.1. Knowledge definition

According to the encoding scheme, the knowledge also consists of two parts, the machine selection probability, and the precedence relation of the operations, denoted by E^t and D^t, respectively, where t denotes the number of iterations. E^t is a $M \times L$ two-dimensional matrix, where L denotes the total number of operations, M is the number of machines, and the element e_{kg} denotes the probability of selecting machine k to process the g-th operation. D^t is a one-dimensional array with length L, where the element d_g denotes the precedence of the g-th operation being processed. The correspondence between operation O_{ij} and the position g is the same as the MS vector in Section 4.2.1.

Table 8
. Response value and significance rank.

Level	*Cr*	*Mu*	*Es*	*Le*	*Ls*	*Ri*
1	**2.758**	**3.022**	**3.214**	3.449	**2.897**	5.202
2	3.430	3.651	3.508	**3.369**	3.278	3.509
3	3.727	4.404	3.900	3.838	2.948	**2.679**
4	3.479	3.613	3.424	3.723	3.565	3.190
5	4.612	3.317	3.960	3.628	5.319	3.427
Delta	1.853	1.382	0.747	0.469	2.422	2.523
Rank	3	4	5	6	2	1

Fig. 16. Main effect plots of significant parameters for the HEA.

4.5.2. Knowledge initialization

The machine selection probability matrix is initialized as shown in the Algorithm 2. Iterate through all operations, and if operation O_{ij} cannot be processed by machine k, then $e0\ kg$ equals to 0; otherwise, $e0\ kg$ is $1/p_{ijk}$. Next, normalize the machine selection probability matrix E^0 (line 8) to ensure that the sum of probability for selecting a machine from its optional machine set is 1 for any operation.

Algorithm 2: Initialize machine selection probabilistic knowledge

Input: Information related to process routes and process times
Output: Initial machine selection probability knowledge matrix E^0
1: Initialize the zero matrix E^0
2: For $g = 1$: L
3: Determine the operation O_{ij} corresponding to position g and its optional machine set M_{ij}
4: For machine k in set M_{ij}
5: $e0\ kg = 1/pt_{ijk}$ // pt_{ijk} represents the processing time of operation O_{ij} processed on machine k
6: End
7: End
8: Normalize matrix E^0

The operation precedence matrix initialization is shown in the Algorithm 3, which consists of two parts. Lines 2–7 and lines 8–10 in Algorithm 3 initialize the matrix for SCOs and SFOs, respectively. For the example in Table 2, the number of SCOs for each job is 2, 3, and 4, respectively. Therefore, for operation O_{11}, $d0\ 1 = 1 \times 4/2 = 2$, for operation O_{32}, $d0\ 7 = 2 \times 4/4 = 2$.

Algorithm 3: Initialize operation precedence knowledge

Input: Information related to process routes
Output: Initial operation precedence knowledge matrix D^0
1: Initialize the zero matrix D^0
2: For $i = 1$: n // n is the number of jobs
3: For $j = 1$: $nc\ i$ // $nc\ i$ is the number of SCOs of job i
4: Determine the position g ($g = \sum_1^{i-1} n_i^c + j$) corresponding to the operation O_{ij}
5: $d0\ g = j \times nc$ max / $nc\ i$ // nc max is the maximize number of SCOs of all jobs
6: End
7: End
8: For $g = \sum_{i=1}^{n} n_i^c + 1$: L
9: $d0\ g$ = random (0,1)
10: End
11: Normalize matrix D^0

4.5.3. Knowledge update

The knowledge update method is shown in Algorithm 4. The machine selection knowledge update method is shown in line 5, where O_{ij} is the operation corresponding to position g, k denotes the machine that processes O_{ij}, and C_{max} is the makespan value of the elite individual. The method of updating the knowledge of operation precedence is shown in line 6. After learning from all the elite individuals, the matrix E^t and D^t are normalized and then fused with $E^{t-1}$and D^{t-1} (lines 10–11 of Algorithm 4) and the normalization is performed again.

Algorithm 4: Update knowledge

Input: Elite individuals X , E^{t-1}, D^{t-1}
Output: Knowledge matrix E^t, D^t
1: Initialize the zero matrix E^t, D^t
2: For x in X
3: For $g = 1$: L
4: Determine the operation O_{ij} corresponding to position g
5: $et\ kg = et\ kg + pt_{ijk}/\ C_{max}$ // pt_{ijk} represents the processing time of operation O_{ij} processed on machine k
6: $d\ t\ kg = pt_{ijk} \times g/\ C_{max}$
7: End
8: End
9: Normalize matrix E^t, D^t
10: $E^t = R \times E^t + (1\text{-}R) \times E^{t-1}$ // R is the learning rate
11: $D^t = R \times D^t + (1\text{-}R) \times D^{t\ -1}$
12: Normalize matrix E^t, D^t

4.5.4. Knowledge-guided individual generation

The acquired knowledge is utilized to generate new individuals to replace those with poor fitness values in the population. The method for generating the MS vector based on the knowledge matrix E^t is described in Algorithm 5. A machine is selected for each operation by generating a random number *rand* between 0 and 1, and then compare it with the cumulative probability of machine selection for that operation.

Algorithm 5: Generate MS vector

Input: Knowledge matrix E^t
Output: MS vector
Initialize an empty vector MS
For $g = 1$: L
$rand$ = random (0, 1)
Calculate the cumulative probability F^t for each operation based on the E^t
For $k = 1$: M

(continued on next page)

Table 9
. Comparison results of MPSFs (time limit = $5\times m\times n\times \max(n_i)/60$ s).

Instances	J-M−C−F	C_{best}					C_{av}						SD					
		IMA	HEA-1	HEA-2	HEA-3	HEA-4	**HEA**	IMA	HEA-1	HEA-2	HEA-3	HEA-4	**HEA**	IMA	HEA-1	HEA-2	HEA-3	HEA-4	**HEA**
MPSF01	3-2-7-2	237	237	237	237	237	**237**	237	237.0	237.0	237.0	237.0	**237.0**	0.00	0.00	0.00	0.00	0.00	**0.00**
MPSF02	3-3-9-3	189	189	189	189	189	**189**	190.5	189.0	189.0	189.0	189.0	**189.0**	1.58	0.00	0.00	0.00	0.00	**0.00**
MPSF03	4-3-11-5	145	144	144	144	144	**144**	145.7	145.3	144.4	144.8	144.8	**144.2**	0.48	0.67	0.52	0.42	0.92	**0.42**
MPSF04	4-5-12-8	165	155	155	155	155	**152**	167.8	159.1	157.6	159.7	157.2	**153.2**	1.69	1.91	2.12	3.56	2.25	**1.55**
MPSF05	5-4-14-6	152	147	147	144	147	**144**	156.3	150.2	148.6	150.5	148.2	**145.5**	2.41	1.48	2.27	3.78	1.93	**1.43**
MPSF06	7-4-14-7	149	148	149	149	149	**148**	152.3	148.9	149.3	149.8	149.3	**148.1**	2.95	0.74	0.48	2.53	0.48	**0.32**
MPSF07	8-5-10-6	102	103	102	102	102	**101**	103.7	103.2	102.9	103.2	102.8	**101.8**	0.82	0.42	0.57	1.03	0.63	**0.42**
MPSF08	8-5-24-8	204	193	196	191	193	**188**	210.8	196.3	201.0	196.1	200.9	**189.5**	5.05	2.00	3.23	3.21	3.38	**1.96**
MPSF09	9-5-26-19	293	294	297	279	297	**269**	305.2	297.9	300.7	287.7	300.8	**274.2**	10.84	3.57	3.62	5.44	4.16	**3.19**
MPSF10	10-5-29-21	604	578	584	576	581	**570**	609.8	589.1	595.0	586.2	589.4	**577.3**	5.85	5.55	5.98	8.22	4.86	**4.22**

IMA: the improved memetic algorithm (IMA) from Gong et al. (2023)
HEA-1: the HEA without reinitialization operator.
HEA-2: the HEA with random reinitialization operator.
HEA-3: the HEA without local search.
HEA-4: the HEA with random local search.

(*continued*)

Algorithm 5: Generate MS vector

```
        If f^t_{k-1,g} < rand <= f^t_{k,g}
          Machine k is selected for operation O_ij corresponding to position g
        break
        End
      End
    End
```

The method for generating the OS vector based on the knowledge matrix D^t is described in Algorithm 6 and an example is given in Fig. 14. The operation precedence knowledge matrix describes the prioritization relation between operations by the numbers in a one-dimensional array. However, the diversity of the generated new individuals is maintained by introducing a random perturbation error Δ (a random array of equal length to the OS vector). The newly generated OS vector can be obtained by sorting the elements of the matrix *Dt new* in non-descending order. The smaller the number corresponding to the operation, the more it should be prioritized for processing. As shown in Fig. 14, in the matrix *Dt new*, the smallest number −1.04 and the largest number 3.07 correspond to codes 2 and 1, respectively. Therefore, in the newly generated OS vector, the first number is 2 and the last number is 1.

Algorithm 6: Generate OS vector

Input: Knowledge matrix D^t
Output: OS vector
Initialize a random vector Δ
$Dt\ new = D^t + \Delta$
Initialize the OS vector according to *Dt new* sorting results

5. Computational experiments

This section is devoted to verifying the effectiveness of the MILP models and HEA algorithm through a total 110 FJSPDS benchmark instances (PSFs 01–100 and MPSFs 01–10), which could be downloaded on line at: https://github.com/98tangler/Jiuqiang-Tang. All computational experiments are solved on a standard PC with an AMD Ryzen 7 4800U CPU and 16 GB of RAM running MS Windows 10. In Section 5.1, the MILP models are solved by Gurobi 9.5.2 solver. In Section 5.2, the design of experiments methodology is utilized to determine the optimal combination of the parameters involved in the algorithms. In Section 5.3, the performance of the proposed operators of the HEA is validated. Finaly, the proposed HEA is compared with the exist method and MILP models to verify the superior effectiveness in Section 5.4. All the proposed and comparative algorithms are coded by MATLAB 2021a.

5.1. Verify the effectiveness of the MILP models

In this section, the proposed MILP models are tested and verified. The MILP model of Gong et al. (2023) is represented as MILP-G. The size complexity of the three models for MPSF and PSFs are listed in Table 5 and Table A.1. The three MILP models are compared in terms of both size complexity and computational complexity. The size complexity is evaluated based on the number of constraints (NCs), the number of integer decision variables (NIVs), and the number of binary decision variables (NBVs). As shown in Table 5 and Table A.1, MILP-2 has much less NCs, NIVs and NBVs than the other two models. MILP-1 has more variables but significantly less NC compared to Model-G.

The computational complexity of the three models for MPSF and PSFs are listed in Table 6 and Table A.2. The computational complexity is evaluated in terms of the optimal solution (C_{max}), the lower bound (LB) value, and the Gap value, which are obtained with a termination criterion of 3600 s. And the notation J-M−C−F denotes the number of jobs, machines, and total number of SCOs and SFOs, respectively. The Gap of PSFs 01–60 solved by different MILP models is shown in Fig. 15. The blue line representing results solved by MILP-2 is almost always at

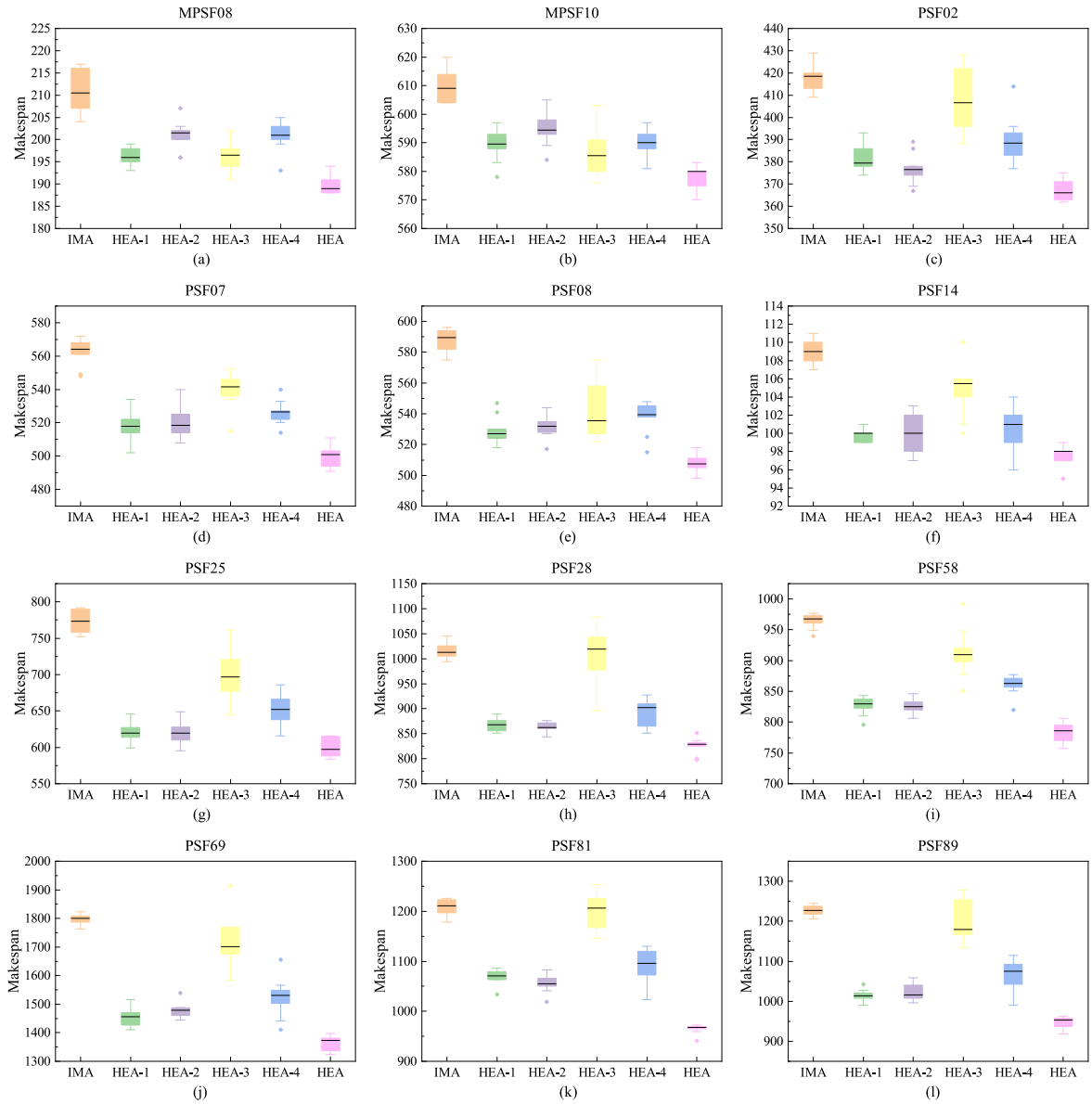


Fig. 17. The box plots for results of IMA, HEA, and variants of HEA.

Table 10
. Comparison results of HEA, IMA and MILP models for MPSFs.

Instances	J-M−C−F	MILP		IMA			HEA		
		C_{opt}	LB	C_{best}	C_{av}	SD	C_{best}	C_{av}	SD
MPSF01	3–2-9–7-2	**237**	237	237	237.0	0.00	**237**	237.0	0.00
MPSF02	3–3-12–9-3	**189**	189	189	189.3	0.95	**189**	189.0	0.00
MPSF03	4–3-16–11-5	**144**	144	144	144.9	0.57	**144**	144.2	0.42
MPSF04	4–5-20–12-8	**152**	152	161	163.9	1.60	**152**	153.2	1.55
MPSF05	5–4-20–14-6	**144**	144	149	152.7	2.31	**144**	145.5	1.43
MPSF06	7–4-21–14-7	**148**	148	149	150.9	1.20	**148**	148.1	0.32
MPSF07	8–5-16–10-6	**101**	101	102	103.0	0.47	**101**	101.8	0.42
MPSF08	8–5-32–24-8	191	144	204	210.3	3.65	**188**	189.5	1.96
MPSF09	9–5-45–26-19	280	145	293	304.6	6.85	**273**	278.5	3.21
MPSF10	10–5-50–29-21	571	413	604	609.1	4.09	**570**	578.2	4.42

the bottom, indicating that its Gap value is always the smallest. As can be seen from Table 6, MILP-2 has the best performance. For small-scale problems, it can obtain the optimal solution in the shortest time. For example, in the case of instance MPSF06, MILP-2 takes nearly 500 s less than MILP-G. For large-scale problems, it is able to obtain better solutions and larger lower bounds. MILP-1 can obtain feasible solutions for some large-scale instances, but MILP-G is not able to find feasible solutions, such as instances PSFs 61–76. Above all, MILP-2 performs best due to having the least number of variables and being able to obtain a better solution in a certain amount of time.

Table 11
. The statistical results of HEA, IMA and MILP models for different scale instances of PSFs.

Instances	MILP		IMA		HEA	
	M_{opt}	M_{lb}	M_{best}	M_{av}	M_{best}	M_{av}
$N \leq 50$ (PSFs 01–05, 15)	290.3	278.7	341.5	351.2	**289.2**	294.9
$N \leq 100$ (PSFs 06–13, 16–27, 35–53)	512.0	410.7	630.7	652.8	**491.3**	504.4
$N \leq 150$ (PSFs 14, 28–34, 54–60)	652.8	448.9	782.9	809.9	**577.7**	594.1
$N \leq 200$ (PSFs 61–66, 76–80)	971.5	801.3	1272.1	1320.1	**916.9**	939.3
$N > 200$ (PSFs 67–75, 81–100)	1168.4	640.0	1169.3	1207.5	**791.5**	815.9

5.2. Parameters setting

The parameters of the HEA contain crossover rate *Cr*, mutation rate *Mu*, elite selection rate *Es*, learning rate *Le*, local search rate *Ls* and reinitialization rate *Ri*, as shown in Table 7. In this section, the Design of Experiment (DOE) method is utilized to set these parameters.

For the instance PSF59, the HEA with each parameter combination is run independently 10 times with a limit of 300 s, and the relative percentage deviation (RPD) is taken as the performance index to indicate the stability of the algorithm.

$$RPD = \frac{C_i - C_{min}}{C_{min}} \times 100 \tag{29}$$

where C_i denotes the best solution obtained in the i-th run of this algorithm and C_{min} denotes the best solution obtained by this algorithm.

The response values of each parameter are shown in Table 8, and the effect of the parameters is shown in Fig. 16. As shown in Table 8, the parameter *Ri* has the largest polarity, indicating that it has the largest impact on the performance, followed by *Ls*, and *Cr*. According to the above analysis, $Cr = 0.95$, $Mu = 0.25$, $Es = 0.25$, $Le = 0.2$, $Ls = 0.5$, and $Ri = 0.15$ are suggested; these values will be used in the following experiments.

5.3. Verify the effectiveness of the operators

To verify the effectiveness of the operators of HEA including the iterative local search operator and knowledge-driven reinitialization operator, four variants and the improved memetic algorithm (IMA) from Gong et al. (2023) are compared with HEA, where HEA-1 is the HEA without reinitialization operator, HEA-2 is the HEA with random reinitialization operator, HEA-3 is the HEA with lack of local search, and HEA-4 is the HEA with random local search. Each algorithm runs independently for each instance 10 times, with a runtime limit of $5 \times m \times n \times \max(n_i)$ milliseconds, where m represents the number of machines, n denotes the number of jobs, and n_i signifies the number of operations of job i. The performance of the algorithm is evaluated using 3 metrics, C_{best}, C_{av} and *SD*, and the results are shown in the Table 9 and Table A.3, where C_{best} refers to the optimal solution, C_{av} refers to the mean value, and *SD* refers to the standard deviation of the solutions obtained in 10 runs. And the notation J-M−C−F denotes the number of jobs, machines, and total number of SCOs and SFOs, respectively. Several box plots of different algorithms solving different instances are given in Fig. 17.

From the data, it can be seen that the proposed HEA algorithm has the best performance. HEA is superior to HEA-1 and HEA-2 in all three metrics, which demonstrates the effectiveness of the knowledge-driven reinitialization strategy proposed. This advantage is because the knowledge-driven reinitialization strategy can learn from the best individuals of the historical population, which makes the reinitialized individuals have good fitness values, and ensures the diversity of the population and avoids premature convergence. HEA significantly outperforms HEA-3 and HEA-4 in all three metrics, which shows the

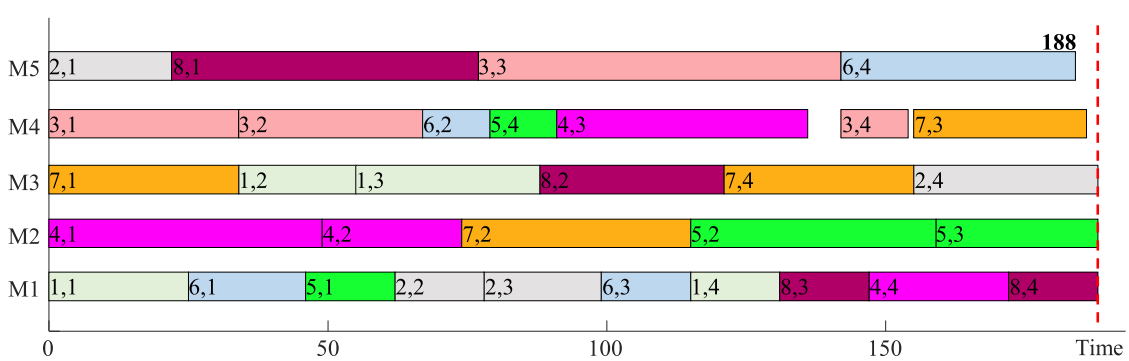


Fig. 18. Gantt chart of the best solution to MPSF08 obtained from HEA (makespan = 188).

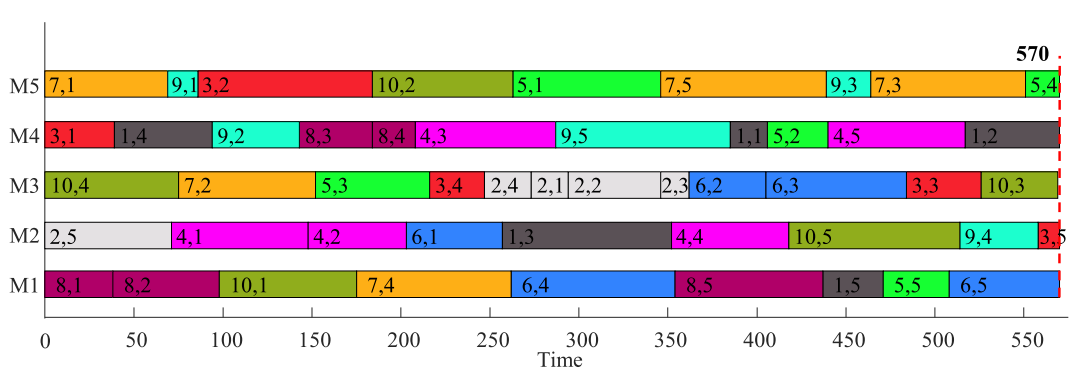


Fig. 19. Gantt chart of the best solution to MPSF10 obtained from HEA (makespan = 570).

effectiveness of the iterative local search operator. This advantage is attributed to the fact that in the limited time, the local search restricts the search scope, effectively improves the quality of the population, and at the same time speeds up the convergence of the algorithm. Comparing the four variants of HEA, it can be seen that the results of HEA-1 and HEA-2 are advantageous compared to HEA-3 and HEA-4, indicating that the iterative local search operator improves the performance of the algorithm most significantly.

5.4. Verify the effectiveness of the HEA

In order to further verify the effectiveness of the proposed HEA, it is compared with the IMA (Gong et al., 2023) and MILP models. The termination condition of algorithms is that the running time reaches 1,000 s or when 100 consecutive iterations fail to optimize performance. The detailed results from 10 independent runs of each algorithm are displayed in Table 10 and Table A.4, where C_{best}, C_{av} and SD denote the best solution, the mean and standard deviation obtained by algorithms, and the C_{opt} and LB represent the optimal solution and the lower bound obtained by MILP models within 3600 s. The statistical results of HEA, IMA and MILP models for different scale instances for PSFs is shown in Table 11, where N is the number of total operations, M_{opt} and M_{lb} represent the average value of the optimal solution and the lower bound obtained by MILP models within 3600 s, M_{best} *and* M_{av} denote the average value of best solution and the mean obtained by algorithms. From Tables 10, 11 and A.4, it can be seen that the performance of HEA is significantly better than others.

Above all, the advantage of HEA can be attributed to the following reasons: First, the improved coding scheme makes the search space and the solution space of the problem match more closely to avoid that some solutions or even possibly even optimal solutions are omitted. Second, the knowledge-driven reinitialization strategy effectively improves the diversity and quality of the population while avoiding the occurrence of premature convergence. Finally, the knowledge-driven reinitialization strategy effectively improves the diversity and quality of the population to prevent the Finally, the application of iterative local search operators effectively improves the relevance of the search and the overall performance of the algorithm. Fig. 18 and Fig. 19 give the Gantt charts corresponding to the optimal solutions of instances MPSF08 and MPSF10.

6. Conclusion

In this paper, two MILP models aiming at minimizing the completion time are proposed for FJSPDS, and their effectiveness is verified through extensive numerical experiments, especially MILP-2 is able to find the optimal solution of the small-scale instances in a shorter time, and the better feasible solution for the large-scale instances as well. For large-scale problems, the HEA that mixes multiple strategies is proposed. Firstly, an improved encoding method is proposed to make the algorithm search space and the problem solution space match more closely, which reduces the possibility of the optimal solution being missed. Secondly, a neighborhood structure is designed according to the characteristics of the SFOs, and the iterative local search operator is proposed by combining another two classical neighborhood structures of FJSP, which effectively improves the search capability. Thirdly, a knowledge-driven reinitialization operator is proposed, which is able to acquire knowledge from the elite individuals in the historical population, and guide the generation of new individuals, ensuring the quality and diversity of the population. Finally, extensive experiments are utilized to verify the effectiveness of the HEA algorithm and its components.

The problem originates from industrial production, which has strong theoretical significance and practical application value, but there are still shortcomings. Firstly, the operation sequence flexibility in the production system is complex and diverse, and more generalized operation sequence flexibility can be further studied. Secondly, other important production resources in the workshop are also limited, and multi-resource joint scheduling optimization should be carried out. Finally, combining techniques such as machine learning improves the algorithm's ability to extract knowledge and its applicability in dynamic optimization.

CRediT authorship contribution statement

Shuai Yuan: Writing – original draft, Visualization, Software, Methodology, Conceptualization. **Xiaomin Zhu:** Writing – original draft, Supervision, Software, Funding acquisition, Formal analysis, Conceptualization. **Wei Cai:** Writing – original draft, Software, Methodology. **Jinsheng Gao:** Writing – original draft, Software, Methodology. **Runtong Zhang:** Supervision, Project administration, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work is supported by the National Natural Science Foundation of China with grant number of 62173025.

Appendix

Table A.1
. The size complexity of MILP models for PSFs.

Instances	MILP-G			MILP-1			MILP-2		
	NCs	NIVs	NBVs	NCs	NIVs	NBVs	NCs	NIVs	NBVs
PSF01	12,449	5624	5454	12,322	5929	5828	**8179**	**4218**	**4117**
PSF02	13,287	6097	5922	13,205	6405	6305	**8710**	**4505**	**4405**
PSF03	13,692	6126	5947	13,469	6448	6347	**8932**	**4589**	**4488**
PSF04	11,230	5123	4962	11,010	5309	5208	**7336**	**3779**	**3678**
PSF05	13,789	6215	6036	13,703	6579	6479	**9029**	**4642**	**4542**
PSF06	23,067	10,327	10,082	22,746	10,920	10,770	**17,268**	**8823**	**8673**
PSF07	25,396	11,693	11,436	25,004	12,159	12,009	**19,016**	**9720**	**9570**
PSF08	27,119	13,095	12,831	26,611	13,262	13,112	**20,006**	**10,251**	**10,101**

(*continued on next page*)

Table A.1 (*continued*)

Instances	MILP-G			MILP-1			MILP-2		
	NCs	NIVs	NBVs	NCs	NIVs	NBVs	NCs	NIVs	NBVs
PSF09	25,764	11,714	11,455	25,405	12,273	12,123	**19,369**	**9886**	**9736**
PSF10	27,066	12,733	12,468	26,857	13,156	13,006	**20,099**	**10,286**	**10,137**
PSF11	17,867	7743	7504	18,027	8417	8334	**11,726**	**5988**	**5906**
PSF12	17,126	7430	7231	17,451	8104	8007	**11,251**	**5735**	**5639**
PSF13	39,261	17,678	17,363	39,514	18,797	18,630	**28,619**	**14,511**	**14,360**
PSF14	53,141	23,988	23,622	53,221	25,380	25,173	**39,118**	**19,790**	**19,586**
PSF15	7786	3232	3112	7853	3619	3559	**4071**	**2108**	**2048**
PSF16	78,421	32,673	32,208	80,204	36,587	36,386	**43,554**	**22,031**	**21,830**
PSF17	77,082	30,318	29,854	78,752	35,149	34,949	**43,417**	**21,920**	**21,720**
PSF18	56,765	26,406	25,951	56,680	27,533	27,332	**41,401**	**21,124**	**20,923**
PSF19	59,866	28,401	27,937	59,606	29,220	29,023	**43,506**	**22,186**	**21,989**
PSF20	54,048	24,780	24,335	53,444	25,792	25,593	**39,395**	**20,083**	**19,884**
PSF21	52,813	24,319	23,879	52,385	25,456	25,255	**38,688**	**19,727**	**19,526**
PSF22	59,380	26,654	26,186	59,366	28,280	28,082	**43,514**	**22,149**	**21,951**
PSF23	62,405	25,201	24,784	63,850	28,792	28,591	**35,402**	**17,905**	**17,704**
PSF24	66,046	27,327	26,900	67,256	30,738	30,539	**36,880**	**18,692**	**18,493**
PSF25	64,945	25,489	25,063	67,008	29,721	29,520	**37,143**	**18,746**	**18,546**
PSF26	70,845	28,509	28,064	72,889	32,741	32,541	**39,928**	**20,179**	**19,980**
PSF27	66,229	27,027	26,598	68,096	30,736	30,536	**37,307**	**18,855**	**18,655**
PSF28	132,504	58,084	57,431	134,147	62,780	62,481	**86,147**	**43,454**	**43,155**
PSF29	160,392	67,796	67,077	162,496	74,446	74,148	**104,239**	**52,503**	**52,205**
PSF30	145,852	62,894	62,206	147,517	68,577	68,276	**95,357**	**48,053**	**47,752**
PSF31	147,380	62,828	62,139	149,560	68,921	68,621	**96,305**	**48,524**	**48,224**
PSF32	127,151	55,583	54,940	127,936	60,158	59,859	**83,454**	**42,079**	**41,780**
PSF33	142,371	60,546	59,868	143,113	66,031	65,735	**92,505**	**46,602**	**46,306**
PSF34	195,526	77,761	77,059	188,002	82,117	81,824	**100,253**	**50,180**	**49,887**
PSF35	53,062	23,390	22,946	53,234	25,223	25,031	**39,259**	**19,985**	**19,798**
PSF36	76,158	31,123	30,663	78,118	35,348	35,147	**42,489**	**21,514**	**21,314**
PSF37	73,072	28,300	27,847	75,938	33,345	33,145	**41,462**	**20,942**	**20,743**
PSF38	74,689	30,261	29,806	76,426	34,470	34,270	**42,065**	**21,259**	**21,061**
PSF39	74,654	30,688	30,233	75,915	34,651	34,450	**41,949**	**21,221**	**21,022**
PSF40	72,091	28,611	28,162	74,890	33,222	33,022	**40,952**	**20,698**	**20,499**
PSF41	75,876	30,322	29,863	77,930	34,935	34,736	**42,769**	**21,608**	**21,409**
PSF42	76,427	30,871	30,410	78,681	35,302	35,102	**42,812**	**21,639**	**21,440**
PSF43	50,373	23,131	22,702	50,111	24,247	24,047	**36,976**	**18,857**	**18,658**
PSF44	72,631	28,476	28,025	74,865	33,152	32,952	**40,944**	**20,694**	**20,494**
PSF45	81,456	35,674	35,207	82,735	38,657	38,457	**44,214**	**22,454**	**22,254**
PSF46	75,080	30,295	29,839	77,394	34,777	34,577	**42,587**	**21,496**	**21,298**
PSF47	71,780	28,571	28,124	73,707	32,913	32,712	**40,411**	**20,415**	**20,215**
PSF48	69,698	28,473	28,033	71,784	32,420	32,219	**39,276**	**19,871**	**19,670**
PSF49	74,468	30,971	30,517	76,321	34,747	34,546	**41,586**	**21,056**	**20,855**
PSF50	82,382	35,069	34,594	84,691	38,902	38,703	**46,114**	**23,347**	**23,148**
PSF51	64,672	26,943	26,520	65,502	30,116	29,915	**36,249**	**18,358**	**18,158**
PSF52	70,189	27,407	26,963	73,082	32,190	31,992	**40,113**	**20,244**	**20,046**
PSF53	64,657	26,192	25,768	65,500	29,667	29,467	**36,336**	**18,358**	**18,158**
PSF54	141,971	61,746	61,067	143,763	67,188	66,891	**92,657**	**46,720**	**46,423**
PSF55	163,854	70,650	39,926	164,590	76,260	75,959	**105,609**	**53,220**	**52,919**
PSF56	133,262	54,217	53,559	134,925	60,886	60,587	**87,881**	**44,180**	**43,881**
PSF57	134,817	56,337	55,676	136,268	62,889	61,989	**88,008**	**44,305**	**44,005**
PSF58	144,102	62,608	61,925	144,860	67,665	67,364	**93,850**	**47,299**	**46,998**
PSF59	133,717	54,975	54,313	136,286	61,785	61,484	**88,386**	**44,472**	**44,171**
PSF60	157,885	67,603	66,889	160,621	73,810	73,511	**102,473**	**51,651**	**51,352**
PSF61	484,037	194,113	192,933	453,784	197,604	197,213	**187,518**	**94,120**	**93,729**
PSF62	472,659	184,475	183,319	435,175	185,643	185,250	**181,061**	**90,775**	**90,382**
PSF63	544,039	211,209	209,955	505,844	215,999	215,607	**212,410**	**106,438**	**106,046**
PSF64	509,059	205,279	204,066	477,946	208,821	208,428	**198,955**	**99,862**	**99,469**
PSF65	519,889	189,734	188,535	468,209	189,406	189,013	**194,007**	**96,969**	**96,576**
PSF66	485,796	187,204	186,022	451,997	192,292	191,900	**188,553**	**94,445**	**94,053**
PSF67	797,245	326,063	324,372	745,639	330,150	329,564	**385,250**	**192,920**	**192,334**
PSF68	901,917	369,326	367,527	842,581	370,916	370,330	**435,589**	**218,130**	**217,544**
PSF69	864,120	357,527	355,760	813,415	361,771	361,185	**423,287**	**211,987**	**211,401**
PSF70	862,549	345,637	343,883	802,594	348,027	347,443	**413,679**	**207,060**	**206,476**
PSF71	921,057	370,783	368,966	859,540	375,042	374,456	**444,131**	**222,373**	**221,787**
PSF72	921,190	376,939	375,126	855,225	376,979	376,392	**441,177**	**220,958**	**220,371**
PSF73	881,330	371,553	369,618	839,182	377,835	377,235	**502,172**	**251,981**	**251,381**
PSF74	795,734	355,798	353,935	775,415	363,475	362,875	**464,863**	**233,600**	**233,000**
PSF75	777,190	334,099	332,268	750,293	343,946	343,347	**450,034**	**225,900**	**225,301**
PSF76	318,410	133,411	132,217	309,952	141,441	141,042	**190,663**	**96,105**	**95,706**
PSF77	336,371	146,627	145,400	329,520	153,116	152,717	**201,432**	**101,613**	**101,214**
PSF78	323,934	134,153	132,952	313,680	141,962	141,562	**192,971**	**97,188**	**96,788**
PSF79	292,179	125,011	123,862	286,071	132,708	132,307	**177,670**	**89,586**	**89,185**
PSF80	303,364	128,314	127,149	295,237	135,611	135,213	**181,946**	**91,721**	**91,323**
PSF81	633,610	274,820	273,017	617,827	285,886	285,286	**435,494**	**218,865**	**218,265**
PSF82	617,109	269,434	267,649	604,562	281,119	280,520	**425,501**	**213,883**	**213,284**

(*continued on next page*)

Table A.1 (*continued*)

Instances	MILP-G			MILP-1			MILP-2		
	NCs	NIVs	NBVs	NCs	NIVs	NBVs	NCs	NIVs	NBVs
PSF83	597,367	258,651	256,900	583,645	270,195	269,596	**411,896**	**207,026**	**206,427**
PSF84	608,355	273,108	271,333	601,030	283,794	283,197	**422,043**	**212,280**	**211,683**
PSF85	607,751	261,370	259,609	586,823	271,132	270,534	**415,846**	**208,910**	**208,312**
PSF86	493,166	199,649	198,294	463,286	204,382	203,933	**246,202**	**123,662**	**123,213**
PSF87	474,022	209,506	208,166	457,541	213,336	212,886	**243,047**	**122,352**	**121,902**
PSF88	458,558	191,050	189,727	442,523	199,246	198,796	**234,974**	**118,114**	**117,664**
PSF89	445,889	182,414	181,118	421,949	188,561	188,111	**226,313**	**113,724**	**113,274**
PSF90	472,444	204,033	202,691	456,485	210,209	209,764	**242,135**	**121,825**	**121,380**
PSF91	408,299	169,972	168,640	391,149	176,783	176,336	**367,633**	**119,539**	**119,092**
PSF92	511,360	216,400	214,897	498,830	227,031	226,550	**301,696**	**151,744**	**151,263**
PSF93	481,467	205,498	204,058	461,252	211,431	210,954	**278,766**	**140,228**	**139,751**
PSF94	460,959	186,071	184,763	434,041	190,910	190,463	**229,790**	**115,325**	**114,878**
PSF95	460,784	192,990	191,670	441,789	199,179	198,728	**234,119**	**117,701**	**117,250**
PSF96	504,950	211,949	210,569	482,293	217,236	216,787	**255,289**	**128,334**	**127,885**
PSF97	499,139	201,791	200,431	469,382	206,851	206,404	**249,156**	**125,048**	**124,601**
PSF98	456,958	188,248	186,943	430,934	192,331	191,885	**229,453**	**115,320**	**114,874**
PSF99	469,605	191,920	190,588	447,593	199,516	199,066	**238,115**	**119,552**	**119,102**
PSF100	538,787	228,498	227,072	517,254	234,579	234,129	**272,800**	**137,180**	**136,730**
Mean	270237.32	112689.45	111534.81	259883.29	117057.5	116734.6	**148361.87**	**73940.29**	**73623.81**

Table A.2
. The computational results of MILP models for PSFs.

Instances	MILP-G				MILP-1				MILP-2			
	C_{max}	Time(s)	Gap (%)	LB	C_{max}	Time(s)	Gap (%)	LB	C_{max}	Time(s)	Gap (%)	LB
PSF01	379	3600	16.89	315	375	3600	16	315	**367**	3600	**7.63**	**339**
PSF02	349	3600	2.87	**339**	351	3600	3.42	**339**	**344**	3600	**1.45**	**339**
PSF03	330	3600	7.88	304	316	3600	3.8	304	**318**	3600	**1.57**	**313**
PSF04	349	3600	11.75	308	348	3600	12.07	306	**341**	**171.41**	–	–
PSF05	333	186.71	–	–	333	45.34	–	–	**333**	**16.42**	–	–
PSF06	**529**	3600	48.96	270	537	3600	48.6	276	536	3600	**41.98**	**311**
PSF07	522	3600	56.71	226	511	3600	48.34	264	**513**	3600	**46.59**	**274**
PSF08	528	3600	35.23	**342**	527	3600	35.1	**342**	**515**	3600	**33.59**	**342**
PSF09	579	3600	64.25	207	576	3600	44.97	**317**	**570**	3600	**44.39**	**317**
PSF10	548	3600	43.07	312	537	3600	41.9	312	**529**	3600	**37.81**	**329**
PSF11	29	3600	27.59	21	**28**	3600	**17.86**	**23**	29	3600	20.69	**23**
PSF12	29	3600	44.83	16	29	3600	31.03	20	**27**	3600	**18.52**	**22**
PSF13	58	3600	63.79	21	59	3600	38.98	36	**57**	3600	**33.33**	**38**
PSF14	104	3600	62.5	39	102	3600	45.1	56	**100**	3600	**44**	56
PSF15	41	3600	2.44	40	41	7.44	–	–	**41**	**0.33**	–	–
PSF16	**836**	3600	58.61	346	873	3600	30.93	603	839	3600	**20.26**	**669**
PSF17	717	3600	54.67	325	674	3600	9.5	610	**670**	**214.51**	–	–
PSF18	534	3600	52.06	256	519	3600	29.87	364	**515**	3600	**29.32**	364
PSF19	471	3600	49.89	236	**458**	3600	44.1	256	464	3600	**37.93**	**288**
PSF20	516	3600	46.51	276	506	3600	36.76	320	**502**	3600	**33.27**	**335**
PSF21	535	3600	52.71	253	536	3600	47.2	283	**530**	3600	**46.6**	**283**
PSF22	539	3600	66.6	180	522	3600	42.53	300	**513**	3600	**41.52**	**300**
PSF23	654	3600	52.14	313	627	3600	24.24	475	**601**	**1359.17**	–	–
PSF24	556	3600	63.31	204	551	3600	20.33	439	**546**	**269.03**	–	–
PSF25	606	3600	59.9	243	582	3600	15.12	494	**554**	**1288.95**	–	–
PSF26	609	3600	42.36	351	605	3600	17.19	**501**	**560**	3600	**10.54**	**501**
PSF27	664	3600	13.86	572	664	3600	13.86	572	**664**	**85.47**	–	–
PSF28	–	3600	–	250	896	3600	53.79	414	**857**	3600	**42.36**	**494**
PSF29	792	3600	79.04	166	729	3600	29.63	513	**726**	3600	**29.34**	**513**
PSF30	871	3600	70.72	255	826	3600	38.86	505	**783**	3600	**34.87**	**510**
PSF31	793	3600	64.82	279	788	3600	32.23	534	**759**	3600	**29.51**	**535**
PSF32	788	3600	70.18	235	812	3600	31.16	559	**751**	3600	**25.57**	**559**
PSF33	163	3600	74.85	41	**151**	3600	36.42	96	**151**	3600	**35.1**	**98**
PSF34	94	3600	78.72	20	87	3600	29.89	61	**79**	3600	**17.72**	**65**
PSF35	103	3600	61.17	40	**98**	3600	43.88	55	100	3600	**42**	**58**
PSF36	600	3600	58.5	249	589	3600	25.3	440	**573**	3600	**10.47**	**513**
PSF37	632	3600	35.76	406	**629**	3600	22.89	485	**629**	**105.68**	–	–
PSF38	576	3600	42.88	329	554	3600	11.73	489	**552**	3600	**8.51**	**505**
PSF39	591	3600	40.27	353	583	3600	18.18	477	**558**	3600	**6.63**	**521**
PSF40	598	3600	32.61	403	574	3600	11.85	506	**565**	3600	**3.54**	**545**
PSF41	599	3600	48.41	309	570	3600	23.86	434	**563**	787	–	–
PSF42	617	3600	47.16	326	586	3600	19.11	474	**561**	3600	**3.39**	**542**
PSF43	**495**	3600	61.21	192	498	3600	**36.95**	314	499	3600	37.07	314
PSF44	624	3600	39.58	377	624	3600	17.95	512	**589**	3600	**5.43**	**557**
PSF45	670	3600	56.42	292	650	3600	37.85	404	**639**	3600	**17.37**	**528**
PSF46	580	3600	41.55	339	576	3600	12.5	504	**549**	3600	**8.2**	**504**

(*continued on next page*)

Table A.2 (*continued*)

Instances	MILP-G				MILP-1				MILP-2			
	C_{max}	Time(s)	Gap (%)	LB	C_{max}	Time(s)	Gap (%)	LB	C_{max}	Time(s)	Gap (%)	LB
PSF47	610	3600	40.82	361	602	3600	10.96	536	**541**	3600	**0.92**	**536**
PSF48	647	3600	2.16	633	633	1428.79	–	–	**633**	**167.36**	–	–
PSF49	584	3600	60.1	233	538	3600	25.28	402	**559**	3600	**16.99**	**464**
PSF50	668	3600	58.98	274	**614**	3600	29.48	433	626	3600	**20.93**	**495**
PSF51	533	3600	53.1	250	542	3600	31	374	**510**	3600	**19.02**	**413**
PSF52	553	3600	44.3	308	514	3600	7.2	477	**496**	3600	**1.21**	**490**
PSF53	617	3600	50.73	304	579	3600	14.34	496	**551**	**1162.26**	–	–
PSF54	853	3600	69.52	260	876	3600	45.43	478	**805**	3600	**34.41**	**528**
PSF55	957	3600	71.06	277	812	3600	34.73	530	**807**	3600	**34.32**	**530**
PSF56	854	3600	71.66	242	815	3600	38.4	502	**786**	3600	**32.95**	**527**
PSF57	898	3600	48.66	461	817	3600	21.66	640	**774**	3600	**17.31**	**640**
PSF58	854	3600	64.52	303	861	3600	49.48	435	**830**	3600	**42.29**	**479**
PSF59	1026	3600	69.3	315	843	3600	34.99	548	**802**	3600	**28.3**	**575**
PSF60	843	3600	72	236	831	3600	24.91	624	**782**	3600	**20.2**	**624**
PSF61	–	3600	–	255	1386	3600	42.71	794	**1236**	3600	**29.69**	**869**
PSF62	–	3600	–	246	1359	3600	17.59	1120	**1153**	3600	**2.86**	**1120**
PSF63	–	3600	–	220	1375	3600	20.58	1092	**1139**	3600	**4.13**	**1092**
PSF64	–	3600	–	118	1309	3600	29.87	918	**1153**	3600	**18.39**	**941**
PSF65	–	3600	–	189	1223	3600	10.22	**1098**	**1165**	3600	**5.75**	**1098**
PSF66	–	3600	–	111	1248	3600	10.5	1117	**1148**	2398.59	–	–
PSF67	–	3600	–	252	–	3600	–	**1172**	**1588**	3600	**26.2**	**1172**
PSF68	–	3600	–	201	–	3600	–	**1070**	–	3600	–	**1070**
PSF69	–	3600	–	150	–	3600	–	**1058**	–	3600	–	**1058**
PSF70	–	3600	–	116	**1896**	3600	**47.57**	**994**	–	3600	–	**994**
PSF71	–	3600	–	118	**1729**	**3600**	**47.48**	**908**	–	3600	–	908
PSF72	–	3600	–	199	–	3600	–	898	**1828**	3600	**50.88**	**898**
PSF73	–	3600	–	90	–	3600	–	322	–	3600	–	322
PSF74	–	3600	–	44	–	3600	–	343	–	3600	–	343
PSF75	–	3600	–	52	–	3600	–	357	–	3600	–	357
PSF76	–	3600	–	124	822	3600	35.52	530	**732**	3600	**27.6**	**530**
PSF77	964	3600	81.54	178	863	3600	46.7	**460**	**798**	3600	**39.72**	481
PSF78	1533	3600	82	276	813	3600	37.15	511	**723**	3600	**29.32**	**511**
PSF79	1404	3600	86.4	191	762	3600	37.01	480	**694**	3600	**30.84**	**480**
PSF80	936	3600	81.84	170	764	3600	24.87	**574**	**746**	3600	**22.92**	575
PSF81	–	3600	–	164	–	3600	–	**532**	–	3600	–	**532**
PSF82	1824	3600	99.04	175	–	3600	–	**501**	**1582**	3600	**68.33**	**501**
PSF83	–	3600	–	219	1933	3600	76.25	459	**1159**	3600	**60.22**	**461**
PSF84	–	3600	–	216	–	3600	–	**492**	–	3600	–	**492**
PSF85	–	3600	–	235	3585	3600	81.19	**531**	–	3600	–	**531**
PSF86	–	3600	–	132	1055	3600	33.27	704	**980**	3600	**25.92**	**726**
PSF87	–	3600	–	108	1156	3600	40.48	688	**1130**	3600	**37.96**	**701**
PSF88	–	3600	–	170	1126	3600	41.47	**659**	**960**	3600	**31.35**	**659**
PSF89	–	3600	–	180	1063	3600	36.78	672	**960**	3600	**24.06**	**729**
PSF90	–	3600	–	151	1012	3600	39.53	612	**1010**	3600	**35.64**	**650**
PSF91	–	3600	–	34	236	3600	50.42	117	**194**	3600	**38.66**	**119**
PSF92	–	3600	–	22	–	3600	–	117	**216**	3600	**42.13**	**125**
PSF93	–	3600	–	25	212	3600	43.87	**119**	**193**	3600	**38.34**	**119**
PSF94	–	3600	–	186	1085	3600	42.49	624	**945**	3600	**32.28**	**640**
PSF95	–	3600	–	154	1078	3600	44.43	599	**944**	3600	**32.1**	**641**
PSF96	–	3600	–	115	1073	3600	36.34	**683**	**954**	3600	**28.41**	**683**
PSF97	1452	3600	86.36	198	948	3600	10.02	**853**	**925**	3600	**7.78**	**853**
PSF98	–	3600	–	112	1020	3600	35.39	659	**977**	3600	**25.59**	**727**
PSF99	–	3600	–	155	1008	3600	19.25	**814**	**931**	3600	**12.57**	**814**
PSF100	–	3600	–	117	1045	3600	33.97	690	**1019**	3600	**27.87**	**735**

Table A.3
. Comparison results of HEA and its variants for PSFs (time limit = $5 \times m \times n \times \max(n_i)/60$ s).

Instance	J-M−C−F	C_{best}						C_{av}						SD					
		IMA	HEA-1	HEA-2	HEA-3	HEA-4	**HEA**	IMA	HEA-1	HEA-2	HEA-3	HEA-4	**HEA**	IMA	HEA-1	HEA-2	HEA-3	HEA-4	**HEA**
PSF01	10–7-33–17	444	404	396	411	400	**388**	455	414.6	409.2	428.3	417.4	**401.7**	4.88	7.99	12.30	14.04	10.01	7.97
PSF02	10–7-30–20	409	374	367	388	377	**362**	417.7	381.9	376.9	408.8	390.2	**367.3**	6.11	6.57	6.67	14.31	10.12	**4.99**
PSF03	10–7-36–14	378	334	334	335	336	**334**	386.2	343.3	344.0	357.5	346.2	**338.1**	4.47	8.30	5.81	19.42	4.76	**3.38**
PSF04	10–7-34–16	419	374	385	381	366	**365**	431	387.2	396.2	406.9	382.9	**373.9**	6.48	10.01	7.89	20.52	11.56	8.56
PSF05	10–7-35–15	366	333	333	338	333	**333**	374.9	342.3	340.4	355.3	343.1	**338.1**	4.86	5.64	4.72	14.24	8.02	**4.36**
PSF06	15–7-54–21	590	537	534	538	542	**511**	597.5	546.2	545.5	582.0	549.7	**521.7**	5.28	7.30	7.47	23.73	5.36	7.13
PSF07	15–7-52–23	548	502	508	515	514	**491**	562.7	519.0	520.2	539.7	525.9	**499.9**	8.26	9.42	8.83	10.49	7.09	**6.47**
PSF08	15–7-43–32	575	518	517	522	515	**498**	587.4	528.9	531.2	543.0	537.8	**508.0**	7.72	8.80	7.02	19.32	10.37	**6.45**
PSF09	15–7-55–20	631	574	568	591	577	**555**	646.9	589.7	588.6	609.1	591.4	**565.1**	8.85	13.22	14.22	11.80	9.50	**7.48**
PSF10	15–7-46–29	585	537	537	537	537	**516**	605.9	553.2	547.3	567.4	546.7	**522.4**	10.68	12.58	8.74	19.10	5.23	**5.19**
PSF11	10–7-42–13	33	30	30	31	29	**28**	33.8	31.0	31.4	32.2	30.6	**29.4**	0.92	0.82	0.70	1.23	1.17	**0.70**
PSF12	10–7-45–13	33	29	30	31	29	**29**	34.4	30.7	31.2	32.9	31.3	**30.3**	0.70	0.95	1.03	1.20	1.34	**0.67**
PSF13	15–7-65–25	64	56	57	59	56	**56**	65	58.5	57.7	61.9	58.5	**57.1**	0.47	1.90	0.95	1.85	1.58	0.57
PSF14	15–7-73–33	107	99	97	100	96	**95**	108.9	99.8	99.8	105.2	100.7	**97.4**	1.20	**0.79**	2.30	3.26	2.31	1.43
PSF15	6–7-27–9	41	41	41	41	41	**41**	41.6	41.0	41.0	41.0	41.0	**41.0**	0.52	0.00	0.00	0.00	0.00	**0.00**
PSF16	10–10-75–25	1082	883	876	1001	886	**842**	1110.9	903.5	904.9	1065.0	932.9	**860.7**	17.85	16.24	16.97	41.04	22.84	**14.28**
PSF17	10–10-80–20	820	701	705	755	717	**694**	863.5	716.9	721.4	812.2	745.8	**710.9**	17.90	10.84	12.04	35.08	17.73	**9.85**
PSF18	20–10-61–39	583	527	524	547	525	**492**	592	538.3	536.2	562.5	540.4	**501.5**	7.21	7.12	9.32	11.79	8.42	**6.67**
PSF19	20–10-60–40	509	456	446	473	449	**427**	521.9	462.7	459.8	502.0	467.4	**433.2**	9.53	5.08	10.09	14.82	9.61	**4.89**
PSF20	20–10-71–29	561	506	502	538	510	**484**	579.5	518.7	515.1	568.9	525.9	**490.2**	10.06	11.85	11.30	30.68	7.87	**4.71**
PSF21	20–10-69–31	570	517	523	529	516	**490**	592.6	536.1	539.9	566.4	535.1	**502.9**	9.06	13.05	12.69	24.63	10.74	9.72
PSF22	20–10-77–23	596	524	515	538	520	**495**	601.9	538.8	534.1	562.9	538.7	**509.1**	4.63	8.50	11.88	17.65	10.98	9.73
PSF23	10–10-74–26	764	625	639	711	658	**624**	803.8	670.1	658.4	764.5	686.3	**643.4**	22.18	22.21	22.54	33.53	13.64	**12.15**
PSF24	10–10-69–31	682	559	550	600	571	**546**	704.4	580.2	577.9	631.9	593.2	**563.3**	11.74	16.26	15.68	16.72	15.61	**7.38**
PSF25	10–10-81–19	752	599	595	645	616	**584**	773.1	621.6	619.9	699.8	651.8	**599.3**	15.88	13.34	16.46	37.97	19.75	**12.31**
PSF26	10–10-77–23	769	621	625	680	631	**585**	784.9	646.7	649.1	719.1	666.3	**606.9**	6.05	23.43	23.48	16.24	16.93	13.23
PSF27	10–10-77–23	763	665	664	674	668	**664**	776	686.2	681.1	721.5	689.0	**674.8**	9.83	17.13	12.73	25.68	13.28	**8.28**
PSF28	15–10-104–46	994	851	844	896	851	**797**	1015.4	867.7	863.1	1010.1	892.9	**825.7**	16.83	13.45	**9.64**	52.04	25.64	16.06
PSF29	15–10-115–35	905	744	766	837	760	**716**	920.3	771.4	782.4	872.2	804.1	**734.2**	9.38	18.67	9.71	35.98	22.72	12.81
PSF30	15–10-112–38	983	822	832	903	828	**787**	997.1	842.1	853.7	965.6	878.5	**805.6**	13.94	11.22	14.10	45.21	25.73	**9.63**
PSF31	15–10-112–38	950	785	776	880	791	**753**	969.2	816.7	817.3	922.5	848.9	**778.2**	13.25	18.67	17.98	34.65	31.04	16.39
PSF32	15–10-107–43	939	804	814	850	806	**756**	974.9	822.4	826.5	948.0	851.3	**772.3**	15.89	18.41	8.13	57.85	23.75	**8.08**
PSF33	15–10-112–38	180	151	154	173	157	**145**	183.8	160.9	159.4	183.8	165.8	**151.2**	2.90	6.35	3.92	7.87	5.75	4.96
PSF34	10–10-122–28	103	89	88	97	87	**82**	106.1	92.6	91.6	103.2	95.9	**86.6**	1.79	3.06	3.44	3.49	4.18	2.84
PSF35	20–10-79–21	108	97	98	103	100	**94**	111	100.7	101.3	109.7	102.8	**95.4**	1.83	2.21	2.06	3.83	1.62	**0.84**
PSF36	10–10-71–29	741	606	614	680	612	**587**	760.3	629.2	627.3	712.8	640.3	**604.3**	13.98	17.10	11.02	26.96	17.12	**9.44**
PSF37	10–10-79–21	720	633	635	693	632	**629**	759.6	640.5	644.7	709.7	657.0	**635.8**	18.42	6.29	9.50	14.86	18.54	**4.21**
PSF38	10–10-77–23	731	587	568	655	577	**564**	741.3	610.4	618.3	684.1	628.4	**583.4**	7.41	16.64	26.02	25.97	23.45	13.52
PSF39	10–10-74–26	720	578	596	663	622	**571**	732.3	615.0	620.0	697.1	642.9	**595.5**	8.03	16.74	16.04	16.91	15.44	13.91
PSF40	10–10-76–24	731	617	620	677	620	**594**	768.2	643.2	640.3	722.3	650.2	**614.5**	16.01	21.55	19.89	24.89	17.45	**11.13**
PSF41	10–10-78–22	708	595	586	643	626	**580**	749.2	621.6	614.6	690.2	642.0	**598.2**	16.86	17.22	18.07	23.90	10.43	**9.85**
PSF42	10–10-77–23	744	597	602	678	**591**	592	762.1	619.4	619.7	709.7	641.6	**606.8**	10.98	20.96	14.64	32.33	20.44	**10.00**
PSF43	20–10-69–31	564	495	480	519	495	**476**	570.5	510.5	507.6	541.9	515.7	**483.6**	3.75	9.77	14.34	17.08	10.76	8.71
PSF44	10–10-77–23	770	629	632	694	660	**612**	794.6	650.0	667.2	725.7	689.0	**626.9**	13.01	12.42	23.81	17.60	15.62	**8.33**
PSF45	10–10-61–39	781	648	649	704	645	**607**	808.2	662.3	661.7	737.9	683.5	**632.2**	11.76	15.46	**8.23**	23.06	17.49	13.16
PSF46	10–10-80–20	764	608	607	670	628	**586**	782.9	630.4	631.6	711.6	650.4	**607.1**	10.94	26.80	21.16	24.59	12.71	14.07
PSF47	10–10-77–23	782	601	615	684	640	**590**	796.7	639.4	634.5	728.7	655.2	**610.6**	10.84	30.86	22.06	23.91	17.15	12.66
PSF48	10–10-74–26	816	678	673	703	669	**653**	847.3	705.8	692.3	758.5	707.6	**678.5**	20.24	17.95	**13.11**	33.10	26.75	15.08
PSF49	10–10-73–27	704	577	563	637	569	**555**	717.3	596.5	582.6	684.4	610.5	**577.5**	8.88	17.88	15.37	25.67	17.00	14.10
PSF50	10–10-70–30	816	639	646	725	684	**622**	826.7	668.8	667.3	766.9	701.5	**647.7**	6.34	15.22	15.27	24.59	8.75	15.36
PSF51	10–10-70–30	615	538	517	576	540	**516**	657.2	555.3	544.2	614.0	567.3	**537.4**	18.83	13.72	14.03	23.27	14.70	**11.64**
PSF52	10–10-80–20	689	518	548	611	578	**510**	712.4	569.9	580.3	658.0	593.3	**553.6**	14.67	28.82	25.62	28.65	**12.60**	21.47
PSF53	10–10-77–23	760	**593**	605	680	608	585	769.3	621.8	633.7	711.7	652.9	**603.6**	9.49	23.53	19.01	22.04	22.17	13.40
PSF54	15–10-108–42	954	823	821	875	850	**767**	969.5	840.8	841.8	940.1	879.4	**785.9**	9.45	13.55	14.92	35.70	14.25	10.41

(*continued on next page*)

Table A.3 (*continued*)

Instance	J-M−C−F	C_{best}						C_{av}						*SD*					
		IMA	HEA-1	HEA-2	HEA-3	HEA-4	**HEA**	IMA	HEA-1	HEA-2	HEA-3	HEA-4	**HEA**	IMA	HEA-1	HEA-2	HEA-3	HEA-4	**HEA**
PSF55	15–10-112–38	913	791	797	846	792	**746**	951.8	824.3	822.2	915.5	847.2	**773.5**	19.01	17.20	**17.18**	31.01	23.68	19.86
PSF56	15–10-126–24	1005	802	819	896	845	**797**	1014.5	839.7	838.2	974.4	877.8	**809.6**	5.19	26.34	10.42	38.53	17.22	11.47
PSF57	15–10-116–34	914	805	793	926	843	**758**	969.2	826.0	822.1	950.7	860.7	**791.6**	24.60	17.05	17.36	23.78	**12.44**	17.26
PSF58	15–10-110–40	940	796	806	851	820	**757**	964.9	826.8	826.5	912.8	860.2	**783.5**	12.07	14.44	**11.09**	38.21	16.34	15.09
PSF59	15–10-121–29	984	797	804	901	840	**779**	1010.9	831.5	831.6	952.9	868.8	**793.7**	13.27	29.35	29.72	35.12	18.47	**7.76**
PSF60	15–10-109–41	917	803	799	881	822	**768**	955.8	825.5	828.9	920.9	855.5	**786.1**	23.30	15.00	20.54	49.40	21.08	**14.19**
PSF61	10–15-136–60	1513	1189	1218	1352	1202	**1185**	1526.7	1228.2	1244.6	1427.0	1304.7	**1202.1**	9.33	22.96	22.68	53.21	44.05	11.22
PSF62	10–15-137–59	1611	1190	1195	1344	1253	**1185**	1670.2	1242.3	1247.7	1462.4	1307.9	**1211.1**	33.74	35.25	31.77	57.69	39.04	**19.10**
PSF63	10–15-149–47	1610	1169	1184	1311	1187	**1169**	1650.1	1205.3	1229.5	1377.2	1277.3	**1186.4**	22.26	22.78	36.55	44.95	37.17	**10.54**
PSF64	10–15-135–61	1489	1177	1181	1272	1217	**1176**	1535	1217.1	1220.3	1364.2	1283.1	**1192.5**	22.58	27.57	34.00	51.65	30.44	**15.14**
PSF65	10–15-161–35	1601	1191	1197	1342	1207	**1179**	1644.3	1240.6	1229.7	1473.9	1274.6	**1223.4**	24.62	39.68	27.35	79.00	37.05	26.36
PSF66	10–15-150–46	1585	1161	1172	1298	1235	**1158**	1617.7	1207.9	1223.2	1390.6	1268.0	**1192.3**	17.51	28.56	23.20	53.16	19.86	18.43
PSF67	15–15-217–76	1814	1443	1387	1669	1440	**1349**	1841.4	1489.2	1474.4	1760.2	1566.6	**1404.9**	21.14	35.00	52.68	66.78	50.41	30.36
PSF68	15–15-219–74	1796	1436	1470	1588	1432	**1340**	1840.6	1477.8	1495.1	1689.7	1566.2	**1381.1**	30.12	29.26	23.79	73.56	67.27	**21.41**
PSF69	15–15-215–78	1763	**1411**	**1445**	**1585**	**1410**	**1324**	1797.8	**1457.3**	**1478.8**	**1713.7**	**1523.1**	**1365.9**	16.51	33.34	25.35	91.50	67.57	26.32
PSF70	15–15-223–70	1779	1442	1457	1651	1460	**1374**	1815.3	1485.4	1476.2	1766.5	1555.9	**1398.0**	25.60	21.74	**15.08**	74.98	54.40	17.69
PSF71	15–15-224–69	1799	1430	1441	1594	1386	**1320**	1830.7	1458.9	1475.6	1757.5	1532.1	**1370.0**	17.78	18.79	25.57	123.64	77.06	25.55
PSF72	15–15-220–73	1743	1382	1427	1584	1404	**1324**	1766.7	1434.4	1474.4	1715.1	1516.6	**1359.2**	19.35	29.78	39.31	91.58	66.86	23.81
PSF73	20–15-221–79	656	558	558	636	548	**503**	666.6	581.0	579.8	679.2	594.0	**526.6**	8.81	12.59	10.36	32.40	25.17	11.31
PSF74	20–15-187–113	661	582	592	662	581	**528**	677.2	606.5	601.2	694.8	623.4	**547.4**	9.87	13.44	**9.28**	34.28	26.65	12.99
PSF75	20–15-213–87	665	584	573	644	579	**540**	676.3	593.3	595.1	691.3	614.1	**555.2**	6.63	9.43	13.40	34.16	24.73	13.31
PSF76	20–15-149–51	916	745	762	899	753	**700**	935.2	777.3	782.8	942.0	811.7	**729.4**	12.02	20.67	18.26	22.58	35.70	17.26
PSF77	20–15-135–65	939	804	803	890	803	**718**	955.1	814.3	811.6	935.6	839.5	**741.4**	11.53	8.74	**7.28**	38.00	21.27	11.74
PSF78	20–15-160–40	968	778	794	878	793	**738**	979	818.3	814.3	971.5	836.7	**764.0**	8.88	21.54	12.04	38.58	27.77	14.64
PSF79	20–15-145–55	892	729	738	867	722	**684**	930.9	754.2	760.5	911.1	774.9	**701.7**	16.24	17.41	12.09	43.34	33.21	**12.24**
PSF80	20–15-148–52	962	783	790	908	783	**734**	981.7	808.5	813.3	974.5	823.7	**760.5**	10.65	19.31	16.74	51.10	28.47	13.27
PSF81	30–15-231–69	1178	1034	1019	1146	1023	**941**	1208.7	1068.8	1055.2	1202.2	1090.3	**964.8**	15.10	14.71	17.59	37.86	37.24	**9.13**
PSF82	30–15-228–72	1282	1136	1111	1201	1122	**1030**	1299.8	1155.4	1149.8	1306.7	1175.9	**1047.5**	9.67	13.62	20.62	57.39	28.84	12.65
PSF83	30–15-231–69	1170	1039	1022	1098	1026	**930**	1197.1	1057.4	1048.5	1214.7	1072.8	**956.6**	11.22	12.63	16.67	71.67	26.90	12.53
PSF84	30–15-206–94	1222	1075	1070	1154	1079	**958**	1236.4	1098.4	1096.7	1226.2	1115.1	**981.4**	8.13	16.63	15.56	54.32	28.65	14.15
PSF85	30–15-244–56	1217	1045	1059	1101	1061	**956**	1244.8	1081.0	1092.0	1247.1	1094.9	**984.6**	16.93	15.44	19.57	77.64	22.75	**12.96**
PSF86	15–15-164–61	1215	999	981	1114	982	**944**	1268	1014.9	1010.1	1168.7	1054.0	**977.2**	23.61	18.17	25.79	53.25	47.51	**17.07**
PSF87	15–15-139–86	1294	1074	1080	1168	1065	**969**	1317.2	1109.2	1117.1	1263.5	1130.7	**986.2**	17.20	16.87	22.61	60.71	43.59	**11.38**
PSF88	15–15-159–66	1191	936	965	1052	946	**859**	1212	973.0	981.9	1130.6	1010.8	**892.4**	15.63	29.80	**11.24**	49.54	47.66	12.49
PSF89	15–15-160–65	1206	990	996	1134	990	**918**	1227	1015.3	1021.8	1201.8	1066.1	**948.4**	12.66	14.13	20.92	51.84	40.57	14.14
PSF90	15–15-146–79	1197	974	988	1112	967	**903**	1212.2	998.4	1018.6	1160.6	1040.9	**933.4**	11.17	26.18	21.46	36.95	45.18	27.78
PSF91	20–15-174–51	248	199	203	243	201	**186**	252.4	207.7	208.8	257.4	218.3	**194.4**	2.50	4.37	3.99	10.96	8.42	3.78
PSF92	20–15-174–66	242	208	207	233	197	**187**	246.3	212.2	211.8	244.4	216.9	**193.5**	3.02	**2.20**	2.78	8.18	9.43	2.55
PSF93	20–15-171–69	224	192	190	223	194	**181**	232.8	197.5	196.9	236.9	203.8	**187.4**	4.10	3.66	3.96	9.76	6.53	**3.06**
PSF94	15–15-175–50	1199	948	953	1111	956	**911**	1219.3	980.5	979.0	1207.5	1029.2	**932.2**	15.34	14.10	14.07	47.01	44.99	**13.39**
PSF95	15–15-156–69	1191	982	985	1045	995	**905**	1232.5	1012.0	1009.1	1144.6	1046.3	**933.7**	18.90	16.19	18.67	51.96	41.41	**14.88**
PSF96	15–15-157–68	1176	941	948	1080	952	**915**	1215.5	996.8	989.5	1118.2	1013.9	**949.9**	18.96	27.96	28.02	29.61	44.41	23.95
PSF97	15–15-174–51	1224	970	967	1114	941	**907**	1241.5	1004.1	1001.1	1196.2	1024.0	**954.0**	13.05	26.75	24.51	48.72	49.85	21.83
PSF98	15–15-159–66	1242	954	982	1137	983	**904**	1262.3	1012.7	1016.3	1188.0	1030.0	**948.2**	17.20	28.47	21.43	30.32	39.08	17.90
PSF99	15–15-172–53	1209	978	960	1085	942	**934**	1249.8	1003.1	994.2	1176.2	1017.2	**964.7**	23.80	21.74	26.14	57.92	56.42	**19.82**
PSF100	15–15-152–73	1250	989	978	1088	972	**931**	1266.5	1008.5	1005.9	1129.0	1021.0	**953.4**	12.41	16.81	15.98	27.97	40.91	14.61
Mean		867.58	714.69	717.13	789.29	721.92	**680.57**	889.0	738.7	739.9	842.1	762.5	**700.5**	12.32	16.21	15.70	34.32	23.43	**11.99**

Table A.4
. Comparison results of HEA, IMA and MILP models for PSFs.

Instances	J-M−C−F	MILP		IMA			HEA		
		C_{opt}	LB	C_{best}	C_{av}	SD	C_{best}	C_{av}	SD
PSF01	10–7-33–17	**367**	339	449	456.4	5.36	**367**	380.1	8.80
PSF02	10–7-30–20	344	339	407	421.6	8.75	**339**	343.4	5.06
PSF03	10–7-36–14	316	313	378	385.6	4.50	**314**	317.7	3.62
PSF04	10–7-34–16	**341**	308	419	430.8	5.47	**341**	349.8	8.66
PSF05	10–7-35–15	**333**	333	355	371.2	9.15	**333**	337.6	4.65
PSF06	15–7-54–21	529	311	568	595.3	13.34	**499**	508.5	7.86
PSF07	15–7-52–23	511	274	548	560.9	6.19	**475**	483.6	6.83
PSF08	15–7-43–32	515	342	575	583.7	3.89	**484**	493.0	7.18
PSF09	15–7-55–20	570	317	631	645.7	6.96	**541**	548.9	6.79
PSF10	15–7-46–29	529	329	585	597.4	6.24	**502**	507.2	5.14
PSF11	10–7-42–13	28	23	33	34.2	0.63	**27**	28.2	0.79
PSF12	10–7-45–13	**27**	22	33	34.1	0.88	**27**	28.1	0.74
PSF13	15–7-65–25	57	38	63	65.3	1.16	**54**	55.0	0.67
PSF14	15–7-73–33	100	56	107	110.0	1.33	**91**	93.1	1.60
PSF15	6–7-27–9	**41**	40	41	41.4	0.52	**41**	41.0	0.00
PSF16	10–10-75–25	836	669	1082	1107.3	15.68	**754**	770.0	13.86
PSF17	10–10-80–20	**670**	610	820	849.1	17.25	**670**	685.4	11.22
PSF18	20–10-61–39	515	364	577	594.3	10.65	**476**	485.0	7.21
PSF19	20–10-60–40	458	288	509	522.8	7.22	**401**	406.4	5.21
PSF20	20–10-71–29	502	335	561	577.7	7.79	**458**	462.4	4.45
PSF21	20–10-69–31	530	283	570	591.7	9.04	**470**	481.5	10.52
PSF22	20–10-77–23	513	300	591	602.5	8.70	**483**	496.2	10.63
PSF23	10–10-74–26	**601**	475	764	799.6	21.34	**601**	619.3	13.43
PSF24	10–10-69–31	**546**	439	668	699.5	13.94	**546**	560.8	8.61
PSF25	10–10-81–19	**554**	494	752	772.2	14.37	**554**	566.2	11.81
PSF26	10–10-77–23	560	501	762	785.1	16.26	**529**	547.8	14.44
PSF27	10–10-77–23	**664**	572	763	775.1	10.81	**664**	674.0	8.94
PSF28	15–10-104–46	857	494	994	1014.4	10.62	**710**	735.9	18.46
PSF29	15–10-115–35	726	513	905	933.9	13.74	**654**	671.3	12.66
PSF30	15–10-112–38	783	510	980	1014.3	18.82	**722**	739.6	11.04
PSF31	15–10-112–38	759	535	931	965.0	19.70	**685**	707.6	18.21
PSF32	15–10-107–43	751	559	939	972.1	15.91	**694**	708.8	9.60
PSF33	15–10-112–38	151	98	180	184.7	2.06	**132**	137.3	5.21
PSF34	10–10-122–28	79	65	103	106.1	1.66	**69**	73.2	3.19
PSF35	20–10-79–21	98	58	108	111.6	1.84	**90**	91.2	0.92
PSF36	10–10-71–29	573	513	734	760.8	14.11	**571**	585.5	10.05
PSF37	10–10-79–21	**629**	485	720	759.0	15.84	**629**	635.4	4.67
PSF38	10–10-77–23	552	505	728	745.8	9.70	**521**	539.5	14.55
PSF39	10–10-74–26	558	521	720	733.5	10.24	**542**	563.8	13.34
PSF40	10–10-76–24	565	545	731	760.9	18.48	**564**	573.7	9.14
PSF41	10–10-78–22	**563**	434	707	749.0	17.58	**563**	578.4	10.70
PSF42	10–10-77–23	561	542	744	763.4	10.44	**554**	566.7	10.73
PSF43	20–10-69–31	495	314	548	570.3	11.72	**448**	454.2	8.69
PSF44	10–10-77–23	589	557	746	792.9	19.12	**580**	592.8	9.22
PSF45	10–10-61–39	639	528	781	799.3	15.39	**576**	599.0	15.41
PSF46	10–10-80–20	549	504	764	786.8	11.67	**512**	532.7	14.66
PSF47	10–10-77–23	541	536	767	800.1	15.57	**536**	555.7	13.84
PSF48	10–10-74–26	**633**	633	803	854.8	21.26	**633**	655.8	17.07
PSF49	10–10-73–27	538	464	701	722.4	11.96	**499**	520.4	15.46
PSF50	10–10-70–30	614	495	804	825.4	14.82	**576**	598.8	17.29
PSF51	10–10-70–30	**510**	413	615	651.6	22.74	**510**	527.0	10.39
PSF52	10–10-80–20	496	490	689	711.5	13.75	**490**	526.1	22.37
PSF53	10–10-77–23	**551**	496	734	766.7	17.20	**551**	566.2	13.37
PSF54	15–10-108–42	805	528	941	975.7	17.17	**711**	727.3	11.61
PSF55	15–10-112–38	807	530	913	952.6	20.85	**687**	710.0	20.54
PSF56	15–10-126–24	786	527	1005	1022.2	10.52	**714**	726.3	11.79
PSF57	15–10-116–34	774	640	914	965.0	31.43	**701**	731.1	20.23
PSF58	15–10-110–40	830	479	940	964.4	12.61	**701**	724.6	17.19
PSF59	15–10-121–29	802	575	974	1008.1	19.69	**706**	719.2	9.04
PSF60	15–10-109–41	782	624	917	959.8	28.05	**689**	706.3	15.03
PSF61	10–15-136–60	1236	869	1507	1558.0	25.50	**1167**	1180.9	11.06
PSF62	10–15-137–59	**1153**	1120	1611	1669.7	27.37	**1153**	1179.1	19.10
PSF63	10–15-149–47	1139	1092	1610	1654.9	21.89	**1131**	1145.7	10.02
PSF64	10–15-135–61	1153	941	1489	1539.3	34.24	**1141**	1152.6	10.74
PSF65	10–15-161–35	**1165**	1098	1601	1656.4	24.47	**1165**	1206.9	29.41
PSF66	10–15-150–46	**1148**	1117	1538	1634.5	52.50	**1148**	1178.2	21.13
PSF67	15–15-217–76	1588	1172	1809	1871.7	31.98	**1187**	1235.2	33.96
PSF68	15–15-219–74	–	1070	1796	1840.9	28.03	**1202**	1229.6	18.55
PSF69	15–15-215–78	–	1058	1763	1810.2	20.77	**1137**	1177.6	28.17
PSF70	15–15-223–70	1896	994	1779	1825.6	23.88	**1164**	1185.8	19.26
PSF71	15–15-224–69	1729	908	1799	1832.1	18.93	**1149**	1192.0	28.84
PSF72	15–15-220–73	1828	898	1709	1777.0	31.98	**1098**	1127.7	25.05

(*continued on next page*)

Table A.4 (*continued*)

Instances	J-M−C−F	MILP		IMA			HEA		
		C_{opt}	LB	C_{best}	C_{av}	SD	C_{best}	C_{av}	SD
PSF73	20–15-221–79	–	322	643	673.2	12.01	**446**	466.3	12.95
PSF74	20–15-187–113	–	343	661	687.2	14.05	**446**	463.6	12.67
PSF75	20–15-213–87	–	357	665	680.3	9.83	**461**	475.9	13.64
PSF76	20–15-149–51	732	530	916	944.6	16.30	**639**	665.6	19.62
PSF77	20–15-135–65	798	481	922	961.6	20.99	**633**	651.9	11.15
PSF78	20–15-160–40	723	511	958	978.5	13.19	**663**	687.0	16.76
PSF79	20–15-145–55	694	480	892	930.2	21.16	**605**	620.3	13.18
PSF80	20–15-148–52	746	575	949	992.9	20.79	**641**	664.6	15.62
PSF81	30–15-231–69	–	532	1178	1218.0	17.98	**910**	917.6	4.53
PSF82	30–15-228–72	1582	501	1267	1310.3	24.41	**971**	985.4	12.77
PSF83	30–15-231–69	1159	461	1170	1201.4	16.53	**900**	917.6	10.38
PSF84	30–15-206–94	–	492	1222	1241.8	12.55	**913**	933.8	15.90
PSF85	30–15-244–56	3585	531	1217	1256.1	16.50	**902**	927.2	15.58
PSF86	15–15-164–61	980	726	1212	1269.6	25.60	**776**	807.4	19.61
PSF87	15–15-139–86	1130	701	1294	1338.4	20.83	**826**	843.2	11.38
PSF88	15–15-159–66	960	659	1191	1228.2	18.27	**745**	774.6	16.17
PSF89	15–15-160–65	960	729	1206	1237.0	17.55	**773**	801.5	16.86
PSF90	15–15-146–79	1010	650	1166	1210.6	20.14	**760**	788.6	29.22
PSF91	20–15-174–51	194	119	248	255.8	3.36	**167**	170.9	2.73
PSF92	20–15-174–66	216	125	242	247.4	2.88	**167**	172.6	3.10
PSF93	20–15-171–69	193	119	224	233.3	4.83	**154**	159.6	3.60
PSF94	15–15-175–50	945	640	1199	1234.5	15.86	**817**	836.9	14.83
PSF95	15–15-156–69	944	641	1191	1253.5	25.49	**778**	802.8	16.86
PSF96	15–15-157–68	954	683	1176	1226.2	28.43	**814**	844.5	26.04
PSF97	15–15-174–51	925	853	1207	1259.0	25.21	**853**	895.1	26.36
PSF98	15–15-159–66	977	727	1216	1267.9	26.70	**804**	845.3	22.42
PSF99	15–15-172–53	931	814	1209	1249.6	32.36	**814**	843.5	21.38
PSF100	15–15-152–73	1019	735	1250	1280.5	19.31	**820**	839.1	15.64
Mean		730.1	518.0	862.9	892.53	15.68	**626.0**	643.45	12.81

Data availability

Data will be made available on request.

References

Alvarez-Valdes, R., Fuertes, A., Tamarit, J.M., Giménez, G., Ramos, R., 2005. A heuristic to schedule flexible job-shop in a glass factory. Eur. J. Oper. Res. Proj. Manage. Schedul. 165, 525–534. https://doi.org/10.1016/j.ejor.2004.04.020.

An, Y., Chen, X., Zhang, J., Li, Y., 2020. A hybrid multi-objective evolutionary algorithm to integrate optimization of the production scheduling and imperfect cutting tool maintenance considering total energy consumption. J. Clean. Prod. 268, 121540. https://doi.org/10.1016/j.jclepro.2020.121540.

An, Y., Chen, X., Gao, K., Zhang, L., Li, Y., Zhao, Z., 2023. A hybrid multi-objective evolutionary algorithm for solving an adaptive flexible job-shop rescheduling problem with real-time order acceptance and condition-based preventive maintenance. Expert Syst. Appl. 212, 118711. https://doi.org/10.1016/j.eswa.2022.118711.

Beach, R., Muhlemann, A.P., Price, D.H.R., Paterson, A., Sharp, J.A., 2000. A review of manufacturing flexibility. Eur. J. Oper. Res. 122, 41–57. https://doi.org/10.1016/S0377-2217(99)00062-4.

Birgin, E.G., Feofiloff, P., Fernandes, C.G., De Melo, E.L., Oshiro, M.T.I., Ronconi, D.P., 2014. A MILP model for an extended version of the Flexible Job Shop Problem. Optim. Lett. 8, 1417–1431. https://doi.org/10.1007/s11590-013-0669-7.

Birgin, E.G., Ferreira, J.E., Ronconi, D.P., 2015. List scheduling and beam search methods for the flexible job shop scheduling problem with sequencing flexibility. Eur. J. Oper. Res. 247, 421–440. https://doi.org/10.1016/j.ejor.2015.06.023.

Burmeister, S.C., Guericke, D., Schryen, G., 2023. A memetic NSGA-II for the multi-objective flexible job shop scheduling problem with real-time energy tariffs. Flex. Serv. Manuf. J. 36, 1530–1570. https://doi.org/10.1007/s10696-023-09517-7.

Cao, Z., Lin, C., Zhou, M., 2021. A Knowledge-Based Cuckoo Search Algorithm to Schedule a Flexible Job Shop With Sequencing Flexibility. IEEE Trans. Autom. Sci. Eng. 18, 56–69. https://doi.org/10.1109/TASE.2019.2945717.

Chen, H., Ihlow, J., Lehmann, C., 1999. Genetic algorithm for flexible job-shop scheduling. In: Proc. 1999 IEEE Int. Conf. Robot. Autom. ICRA99 May 10 1999 - May 15 1999 2, pp. 1120–1125.

Chen, X., Li, J., Du, Y., 2023b. A hybrid evolutionary immune algorithm for fuzzy flexible job shop scheduling problem with variable processing speeds. Expert Syst. Appl. 233, 120891. https://doi.org/10.1016/j.eswa.2023.120891.

Chen, B., Zhang, J., Long, T., Hou, Y., Jiang, S., 2023a. The aircraft workpiece paint shop scheduling problem: A case study. J. Manuf. Syst. 68, 426–442. https://doi.org/10.1016/j.jmsy.2023.05.003.

Dauzère-Pérès, S., Ding, J., Shen, L., Tamssaouet, K., 2024. The flexible job shop scheduling problem: A review. Eur. J. Oper. Res. 314, 409–432. https://doi.org/10.1016/j.ejor.2023.05.017.

Destouet, C., Tlahig, H., Bettayeb, B., Mazari, B., 2023. Flexible job shop scheduling problem under Industry 5.0: A survey on human reintegration, environmental consideration and resilience improvement. J. Manuf. Syst. 67, 155–173. https://doi.org/10.1016/j.jmsy.2023.01.004.

Duan, J., Wang, J., 2021. Energy-efficient scheduling for a flexible job shop with machine breakdowns considering machine idle time arrangement and machine speed level selection. Comput. Ind. Eng. 161, 107677. https://doi.org/10.1016/j.cie.2021.107677.

Duan, J., Wang, J., 2022. Robust scheduling for flexible machining job shop subject to machine breakdowns and new job arrivals considering system reusability and task recurrence. Expert Syst. Appl. 203, 117489. https://doi.org/10.1016/j.eswa.2022.117489.

Fan, J., Shen, W., Gao, L., Zhang, C., Zhang, Z., 2021. A hybrid Jaya algorithm for solving flexible job shop scheduling problem considering multiple critical paths. J. Manuf. Syst. 60, 298–311. https://doi.org/10.1016/j.jmsy.2021.05.018.

Gao, K., Cao, Z., Zhang, L., Chen, Z., Han, Y., Pan, Q., 2019. A review on swarm intelligence and evolutionary algorithms for solving flexible job shop scheduling problems. IEEECAA J. Autom. Sin. 6, 904–916. https://doi.org/10.1109/JAS.2019.1911540.

Ghaedy-Heidary, E., Nejati, E., Ghasemi, A., Torabi, S.A., 2024. A simulation optimization framework to solve Stochastic Flexible Job-Shop Scheduling Problems—Case: Semiconductor manufacturing. Comput. Oper. Res. 163, 106508. https://doi.org/10.1016/j.cor.2023.106508.

Gong, G., Deng, Q., Gong, X., Huang, D., 2021. A non-dominated ensemble fitness ranking algorithm for multi-objective flexible job-shop scheduling problem considering worker flexibility and green factors. Knowl.-Based Syst. 231, 107430. https://doi.org/10.1016/j.knosys.2021.107430.

Gong, G., Tang, J., Huang, D., Luo, Q., Zhu, K., Peng, N., 2023. Energy-efficient flexible job shop scheduling problem considering discrete operation sequence flexibility. Swarm Evol. Comput. 84, 101421. https://doi.org/10.1016/j.swevo.2023.101421.

Guo, Z., Wei, L., Zhang, J., Hu, Z., Sun, H., Che, H., 2024. Multi-level guided evolution algorithm for solving fuzzy flexible job shop problem. Appl. Soft Comput. 163, 111932. https://doi.org/10.1016/j.asoc.2024.111932.

Huang, X.W., Zhao, X.Y., Ma, X.L., 2014. An improved genetic algorithm for job-shop scheduling problem with process sequence flexibility. Int. J. Simul. Model. 13, 510–522. https://doi.org/10.2507/IJSIMM13(4)CO20.

Ji, B., Zhang, S., Yu, S.S., Xiao, X., Chen, C., Zheng, G., 2023. Novel model and solution method for flexible job shop scheduling problem with batch processing machines. Comput. Oper. Res. 161, 106442. https://doi.org/10.1016/j.cor.2023.106442.

Jia, S., Hu, Z.-H., 2014. Path-relinking Tabu search for the multi-objective flexible job shop scheduling problem. Comput. Oper. Res. 47, 11–26. https://doi.org/10.1016/j.cor.2014.01.010.

Jiang, Z., Yuan, S., Ma, J., Wang, Q., 2021. The evolution of production scheduling from Industry 3.0 through Industry 4.0. Int. J. Prod. Res. 60, 3534–3554. https://doi.org/10.1080/00207543.2021.1925772.

Kasapidis, G.A., Paraskevopoulos, D.C., Repoussis, P.P., Tarantilis, C.D., 2021. Flexible job shop scheduling problems with arbitrary precedence graphs. Prod. Oper. Manag. 30, 4044–4068. https://doi.org/10.1111/poms.13501.

Li, R., Gong, W., Lu, C., 2022a. Self-adaptive multi-objective evolutionary algorithm for flexible job shop scheduling with fuzzy processing time. Comput. Ind. Eng. 168, 108099. https://doi.org/10.1016/j.cie.2022.108099.

Li, X., Guo, X., Tang, H., Wu, R., Wang, L., Pang, S., Liu, Z., Xu, W., Li, X., 2022b. Survey of integrated flexible job shop scheduling problems. Comput. Ind. Eng. 174, 108786. https://doi.org/10.1016/j.cie.2022.108786.

Liang, J., Ban, X., Yu, K., Qu, B., Qiao, K., Yue, C., Chen, K., Tan, K.C., 2023. A survey on evolutionary constrained multiobjective optimization. IEEE Trans. Evol. Comput. 27, 201–221. https://doi.org/10.1109/TEVC.2022.3155533.

Lin, C., Cao, Z., Zhou, M., 2023. Learning-based cuckoo search algorithm to schedule a flexible job shop with sequencing flexibility. IEEE Trans. Cybern. 53, 6663–6675. https://doi.org/10.1109/TCYB.2022.3210228.

Liu, M., Lv, J., Du, S., Deng, Y., Shen, X., Zhou, Y., 2024a. Multi-resource constrained flexible job shop scheduling problem with fixture-pallet combinatorial optimisation. Comput. Ind. Eng. 188, 109903. https://doi.org/10.1016/j.cie.2024.109903.

Liu, Z., Wang, J., Zhang, C., Chu, H., Ding, G., Zhang, L., 2021. A hybrid genetic-particle swarm algorithm based on multilevel neighbourhood structure for flexible job shop scheduling problem. Comput. Oper. Res. 135, 105431. https://doi.org/10.1016/j.cor.2021.105431.

Liu, Z., Zha, J., Yan, J., Zhang, Y., Zhao, T., Cheng, Q., Cheng, C., 2024b. An improved genetic algorithm with an overlapping strategy for solving a combination of order batching and flexible job shop scheduling problem. Eng. Appl. Artif. Intell. 127, 107321. https://doi.org/10.1016/j.engappai.2023.107321.

Liu, N., Zhang, Y.F., Lu, W.F., 2019. Improving energy efficiency in discrete parts manufacturing system using an ultra-flexible job shop scheduling algorithm. Int. J. Precis. Eng. Manuf.-Green Technol. 6, 349–365. https://doi.org/10.1007/s40684-019-00055-y.

Lu, X., Lu, C., 2024. Mixed-production flexible assembly job shop scheduling considering parallel assembly sequence variations under dual-resource constraints using multi-objective hybrid memetic algorithm. Comput. Oper. Res. 106932. https://doi.org/10.1016/j.cor.2024.106932.

Neumann, A., Hajji, A., Rekik, M., Pellerin, R., 2023. Genetic algorithms for planning and scheduling engineer-to-order production: a systematic review. Int. J. Prod. Res. 62, 2888–2917. https://doi.org/10.1080/00207543.2023.2237122.

Pan, Z., Lei, D., Wang, L., 2022. A Bi-population evolutionary algorithm with feedback for energy-efficient fuzzy flexible job shop scheduling. IEEE Trans. Syst. Man Cybern. Syst. 52, 5295–5307. https://doi.org/10.1109/TSMC.2021.3120702.

Saouabi, M.D.E., Nouri, H.E., Belkahla Driss, O., 2024. A two-level evolutionary algorithm for dynamic scheduling in flexible job shop environment. Evol. Intell. 17, 4133–4153. https://doi.org/10.1007/s12065-024-00976-x.

Schworm, P., Wu, X., Klar, M., Glatt, M., Aurich, J.C., 2024. Multi-objective Quantum Annealing approach for solving flexible job shop scheduling in manufacturing. J. Manuf. Syst. 72, 142–153. https://doi.org/10.1016/j.jmsy.2023.11.015.

Shao, S., Xu, G., Li, J., Liu, Z., Jin, Z., 2025. A job assignment scheduling algorithm with variable sublots for lot-streaming flexible job shop problem based on NSGAII. Comput. Oper. Res. 173, 106866. https://doi.org/10.1016/j.cor.2024.106866.

Shi, Y., Zhang, G., Gao, L., Yuan, K., 2009. A novel initialization method for solving flexible job-shop scheduling problem. In: 2009 International Conference on Computers & Industrial Engineering. Presented at the Industrial Engineering (CIE39). IEEE, Troyes, France, pp. 68–73, 10.1109/ICCIE.2009.5223891.

Sun, J., Zhang, G., Lu, J., Zhang, W., 2021. A hybrid many-objective evolutionary algorithm for flexible job-shop scheduling problem with transportation and setup times. Comput. Oper. Res. 132, 105263. https://doi.org/10.1016/j.cor.2021.105263.

Sun, K., Zheng, D., Song, H., Cheng, Z., Lang, X., Yuan, W., Wang, J., 2023. Hybrid genetic algorithm with variable neighborhood search for flexible job shop scheduling problem in a machining system. Expert Syst. Appl. 215, 119359. https://doi.org/10.1016/j.eswa.2022.119359.

Tamssaouet, K., Dauzère-Pérès, S., Knopp, S., Bitar, A., Yugma, C., 2021. Multiobjective optimization for complex flexible job-shop scheduling problems. Eur. J. Oper. Res. 296, 87–100. https://doi.org/10.1016/j.ejor.2021.03.069.

Tang, J., Gong, G., Peng, N., Zhu, K., Huang, D., Luo, Q., 2024. An effective memetic algorithm for distributed flexible job shop scheduling problem considering integrated sequencing flexibility. Expert Syst. Appl. 242, 122734. https://doi.org/10.1016/j.eswa.2023.122734.

Tutumlu, B., Saraç, T., 2023. A MIP model and a hybrid genetic algorithm for flexible job-shop scheduling problem with job-splitting. Comput. Oper. Res. 155, 106222. https://doi.org/10.1016/j.cor.2023.106222.

Usman, S., Lu, C., Gao, G., 2024. Flexible job-shop scheduling with limited flexible workers using an improved multiobjective discrete teaching–learning based optimization algorithm. Optim. Eng. 25, 1237–1270. https://doi.org/10.1007/s11081-023-09842-8.

Vilcot, G., Billaut, J.-C., 2008. A tabu search and a genetic algorithm for solving a bicriteria general job shop scheduling problem. Eur. J. Oper. Res. 190, 398–411. https://doi.org/10.1016/j.ejor.2007.06.039.

Vital-Soto, A., Azab, A., Baki, M.F., 2020. Mathematical modeling and a hybridized bacterial foraging optimization algorithm for the flexible job-shop scheduling problem with sequencing flexibility. J. Manuf. Syst. 54, 74–93. https://doi.org/10.1016/j.jmsy.2019.11.010.

Vital-Soto, A., Baki, M.F., Azab, A., 2023. A multi-objective mathematical model and evolutionary algorithm for the dual-resource flexible job-shop scheduling problem with sequencing flexibility. Flex. Serv. Manuf. J. 35, 626–668. https://doi.org/10.1007/s10696-022-09446-x.

Weckenborg, C., Schumacher, P., Thies, C., Spengler, T.S., 2024. Flexibility in manufacturing system design: A review of recent approaches from Operations Research. Eur. J. Oper. Res. 315, 413–441. https://doi.org/10.1016/j.ejor.2023.08.050.

Xiao, B., Zhao, Z., Wu, Y., Zhu, X., Peng, S., Su, H., 2024. An improved MOEA/D for multi-objective flexible job shop scheduling by considering efficiency and cost. Comput. Oper. Res. 167, 106674. https://doi.org/10.1016/j.cor.2024.106674.

Xu, H., Bao, Z.R., Zhang, T., 2017. Solving dual flexible job-shop scheduling problem using a Bat Algorithm. Adv. Prod. Eng. Manag. 12, 5–16. https://doi.org/10.14743/apem2017.1.235.

Xu, Y., Zhang, M., Yang, M., Wang, D., 2024. Hybrid quantum particle swarm optimization and variable neighborhood search for flexible job-shop scheduling problem. J. Manuf. Syst. 73, 334–348. https://doi.org/10.1016/j.jmsy.2024.02.007.

Yao, Y.-J., Liu, Q.-H., Li, X.-Y., Gao, L., 2023. A novel MILP model for job shop scheduling problem with mobile robots. Robot. Comput.-Integr. Manuf. 81, 102506. https://doi.org/10.1016/j.rcim.2022.102506.

Yu, L., Zhu, C., Shi, J., Zhang, W., 2017. An extended flexible job shop scheduling model for flight deck scheduling with priority, parallel operations, and sequence flexibility. Sci. Program. 2017, 2463252. https://doi.org/10.1155/2017/2463252.

Zhang, J., Wei, L., Guo, Z., Sun, H., Hu, Z., 2023. A survey of meta-heuristic algorithms in optimization of space scale expansion. Swarm Evol. Comput. 84, 101462. https://doi.org/10.1016/j.swevo.2023.101462.

Zhou, Y., Du, S., Liu, M., Shen, X., 2024. Machine-fixture-pallet resources constrained flexible job shop scheduling considering loading and unloading times under pallet automation system. J. Manuf. Syst. 73, 143–158. https://doi.org/10.1016/j.jmsy.2024.01.010.